

Recent developments in the PySCF program package

Cite as: J. Chem. Phys. **153**, 024109 (2020); <https://doi.org/10.1063/5.0006074>

Submitted: 27 February 2020 . Accepted: 15 June 2020 . Published Online: 09 July 2020

Qiming Sun , Xing Zhang , Samraghi Banerjee, Peng Bao, Marc Barbry , Nick S. Blunt , Nikolay A. Bogdanov, George H. Booth , Jia Chen , Zhi-Hao Cui , Janus J. Eriksen , Yang Gao , Sheng Guo , Jan Hermann , Matthew R. Hermes, Kevin Koh , Peter Koval , Susi Lehtola , Zhendong Li , Junzi Liu, Narbe Mardirossian, James D. McClain, Mario Motta , Bastien Mussard , Hung Q. Pham , Artem Pulkin , Wirawan Purwanto , Paul J. Robinson , Enrico Ronca , Elvira R. Sayfutyarova , Maximilian Scheurer , Henry F. Schurkus , James E. T. Smith , Chong Sun , Shi-Ning Sun, Shiv Upadhyay , Lucas K. Wagner , Xiao Wang , Alec White , James Daniel Whitfield , Mark J. Williamson , Sebastian Wouters, Jun Yang , Jason M. Yu , Tianyu Zhu , Timothy C. Berkelbach , Sandeep Sharma, Alexander Yu. Sokolov , and Garnet Kin-Lic Chan 

COLLECTIONS

Paper published as part of the special topic on [Electronic Structure Software](#)

Note: This article is part of the JCP Special Topic on Electronic Structure Software.



[View Online](#)



[Export Citation](#)



[CrossMark](#)

ARTICLES YOU MAY BE INTERESTED IN

[PSI4 1.4: Open-source software for high-throughput quantum chemistry](#)

The Journal of Chemical Physics **152**, 184108 (2020); <https://doi.org/10.1063/5.0006002>

[Coupled-cluster techniques for computational chemistry: The CFOUR program package](#)

The Journal of Chemical Physics **152**, 214108 (2020); <https://doi.org/10.1063/5.0004837>

[Dalton Project: A Python platform for molecular- and electronic-structure simulations of complex systems](#)

The Journal of Chemical Physics **152**, 214115 (2020); <https://doi.org/10.1063/1.5144298>

Lock-in Amplifiers
up to 600 MHz



Recent developments in the PySCF program package

Cite as: J. Chem. Phys. 153, 024109 (2020); doi: 10.1063/5.0006074

Submitted: 27 February 2020 • Accepted: 15 June 2020 •

Published Online: 9 July 2020



Qiming Sun,¹ Xing Zhang,² Samraghi Banerjee,³ Peng Bao,⁴ Marc Barbry,⁵ Nick S. Blunt,⁶ Nikolay A. Bogdanov,⁷ George H. Booth,⁸ Jia Chen,^{9,10} Zhi-Hao Cui,² Janus J. Eriksen,¹¹ Yang Gao,¹² Sheng Guo,¹³ Jan Hermann,^{14,15} Matthew R. Hermes,¹⁶ Kevin Koh,¹⁷ Peter Koval,¹⁸ Susi Lehtola,¹⁹ Zhendong Li,²⁰ Junzi Liu,²¹ Narbe Mardirossian,²² James D. McClain,²³ Mario Motta,²⁴ Bastien Mussard,²⁵ Hung Q. Pham,¹⁶ Artem Pulkin,²⁶ Wirawan Purwanto,²⁷ Paul J. Robinson,²⁸ Enrico Ronca,²⁹ Elvira R. Sayfutyarova,³⁰ Maximilian Scheurer,³¹ Henry F. Schurkus,² James E. T. Smith,²⁵ Chong Sun,² Shi-Ning Sun,¹² Shiv Upadhyay,³² Lucas K. Wagner,³³ Xiao Wang,³⁴ Alec White,² James Daniel Whitfield,³⁵ Mark J. Williamson,³⁶ Sebastian Wouters,³⁷ Jun Yang,³⁸ Jason M. Yu,³⁹ Tianyu Zhu,² Timothy C. Berkelbach,^{28,34} Sandeep Sharma,²⁵ Alexander Yu. Sokolov,³ and Garnet Kin-Lic Chan^{2,a}

AFFILIATIONS

¹ AxiomQuant Investment Management LLC, Shanghai 200120, China

² Division of Chemistry and Chemical Engineering, California Institute of Technology, Pasadena, California 91125, USA

³ Department of Chemistry and Biochemistry, The Ohio State University, Columbus, Ohio 43210, USA

⁴ Beijing National Laboratory for Molecular Sciences, State Key Laboratory for Structural Chemistry of Unstable and Stable Species, Institute of Chemistry, Chinese Academy of Sciences, Beijing 100190, China

⁵ Simbeyond B.V., P.O. Box 513, NL-5600 MB, Eindhoven, The Netherlands

⁶ Department of Chemistry, Lensfield Road, Cambridge CB2 1EW, United Kingdom

⁷ Max Planck Institute for Solid State Research, Heisenbergstraße 1, 70569 Stuttgart, Germany

⁸ Department of Physics, King's College London, Strand, London WC2R 2LS, United Kingdom

⁹ Department of Physics, University of Florida, Gainesville, Florida 32611, USA

¹⁰ Quantum Theory Project, University of Florida, Gainesville, Florida 32611, USA

¹¹ School of Chemistry, University of Bristol, Cantock's Close, Bristol BS8 1TS, United Kingdom

¹² Division of Engineering and Applied Science, California Institute of Technology, Pasadena, California 91125, USA

¹³ Google Inc., Mountain View, California 94043, USA

¹⁴ FU Berlin, Department of Mathematics and Computer Science, Arnimallee 6, 14195 Berlin, Germany

¹⁵ TU Berlin, Machine Learning Group, Marchstr. 23, 10587 Berlin, Germany

¹⁶ Department of Chemistry, Chemical Theory Center, and Supercomputing Institute, University of Minnesota, 207 Pleasant Street SE, Minneapolis, Minnesota 55455, USA

¹⁷ Department of Chemistry and Biochemistry, The University of Notre Dame du Lac, 251 Nieuwland Science Hall, Notre Dame, Indiana 46556, USA

¹⁸ Simune Atomistics S.L., Avenida Tolosa 76, Donostia-San Sebastian, Spain

¹⁹ Department of Chemistry, University of Helsinki, P.O. Box 55 (A. I. Virtasen aukio 1), FI-00014 Helsinki, Finland

²⁰ Key Laboratory of Theoretical and Computational Photochemistry, Ministry of Education, College of Chemistry, Beijing Normal University, Beijing 100875, China

²¹ Department of Chemistry, The Johns Hopkins University, Baltimore, Maryland 21218, USA

²² AMGEN Research, One Amgen Center Drive, Thousand Oaks, California 91320, USA

²³ DRW Holdings LLC, Chicago, Illinois 60661, USA

²⁴ IBM Almaden Research Center, San Jose, California 95120, USA

²⁵Department of Chemistry, University of Colorado, Boulder, Colorado 80302, USA

²⁶QuTech and Kavli Institute of Nanoscience, Delft University of Technology, The Netherlands

²⁷Information Technology Services, Old Dominion University, Norfolk, Virginia 23529, USA

²⁸Department of Chemistry, Columbia University, New York, New York 10027, USA

²⁹Istituto per i Processi Chimico Fisici del CNR (IPCF-CNR), Via G. Moruzzi, 1, 56124 Pisa, Italy

³⁰Department of Chemistry, Yale University, 225 Prospect Street, New Haven, Connecticut 06520, USA

³¹Interdisciplinary Center for Scientific Computing, Ruprecht-Karls University of Heidelberg, 205 Im Neuenheimer Feld, 69120 Heidelberg, Germany

³²Department of Chemistry, University of Pittsburgh, Pittsburgh, Pennsylvania 15260, USA

³³Department of Physics and Institute for Condensed Matter Theory, University of Illinois at Urbana-Champaign, Illinois 61801, USA

³⁴Center for Computational Quantum Physics, Flatiron Institute, New York, New York 10010, USA

³⁵Department of Physics and Astronomy, Dartmouth College, Hanover, New Hampshire 03755, USA

³⁶Department of Chemistry, University of Cambridge, Lensfield Road, Cambridge CB2 1EW, United Kingdom

³⁷Bricsys NV, Bellevue 5/201, 9050 Gent, Belgium

³⁸Department of Chemistry, The University of Hong Kong, Pokfulam Road, Hong Kong SAR, China

³⁹Department of Chemistry, University of California, Irvine, 1102 Natural Sciences II, Irvine, California 92697-2025, USA

Note: This article is part of the JCP Special Topic on Electronic Structure Software.

a) Author to whom correspondence should be addressed: gkc1000@gmail.com

ABSTRACT

PySCF is a Python-based general-purpose electronic structure platform that supports first-principles simulations of molecules and solids as well as accelerates the development of new methodology and complex computational workflows. This paper explains the design and philosophy behind PySCF that enables it to meet these twin objectives. With several case studies, we show how users can easily implement their own methods using PySCF as a development environment. We then summarize the capabilities of PySCF for molecular and solid-state simulations. Finally, we describe the growing ecosystem of projects that use PySCF across the domains of quantum chemistry, materials science, machine learning, and quantum information science.

Published under license by AIP Publishing. <https://doi.org/10.1063/5.0006074>

I. INTRODUCTION

This article describes the current status of the Python Simulations of Chemistry Framework, also known as PySCF, as of version 1.7.1. The PySCF project was originally started in 2014 by Sun, then in the group of Chan, in the context of developing a tool to enable *ab initio* quantum embedding calculations. However, it rapidly outgrew its rather specialized roots to become a general purpose development platform for quantum simulations and electronic structure theory. The early history of PySCF is recounted in Ref. 1. Now, PySCF is a production ready tool that implements many of the most commonly used methods in molecular quantum chemistry and solid-state electronic structure. Since its inception, PySCF has been a free and open-source package hosted on Github² and is now also available through pip,³ conda,⁴ and a number of other distribution platforms. It has a userbase numbering in the hundreds and over 60 code contributors. Beyond chemistry and materials science, it has also found use in the areas of data science,^{5,6} machine learning,^{7–13} and quantum computing^{14–16} in both academia and industry. To mark its transition from a code developed by a single group to a broader community effort, the leadership of PySCF was expanded in 2019 to a board of directors.¹⁷

While the fields of quantum chemistry and solid-state electronic structure are rich with excellent software,^{18–25} the development of PySCF is guided by some unique principles in the order of priority as follows:

1. PySCF should be more than a computational tool; it should be a development platform. We aim for users to be empowered to modify the code, implement their own methods without the assistance of the original developers, and incorporate parts of the code in a modular fashion into their own projects.
2. Unlike many packages that focus on either molecular chemistry or materials science applications, PySCF should support both equally, allowing calculations on molecules and materials to be carried out in the same numerical framework and with the same theoretical approximations.
3. PySCF should enable users outside of the chemical sciences (such as workers in machine learning and quantum information theory) to carry out quantum chemistry simulations.

In the remainder of this article, we elaborate on these guiding principles of PySCF, describing how they have impacted the program design and implementation and how they can be used

to implement a new functionality in new projects. We provide a brief summary of the implemented methods and conclude with an overview of the PySCF ecosystem in different areas of science.

II. THE DESIGN PHILOSOPHY BEHIND PySCF

All quantum simulation workflows naturally require some level of programming and customization. This may arise in simple tasks, such as scanning a potential energy surface, tabulating results, or automating input generation, or in more advanced use cases that include more substantial programming, such as with complex data processing, incorporating logic into the computational workflow, or when embedding customized algorithms into the computation. In either case, the ability to program with and extend one's simulation software greatly empowers the user. PySCF is designed to serve as a basic program library that can facilitate custom computational tasks and workflows as well as form the starting point for the development of new algorithms.

To enable this, PySCF is constructed as a library of modular components with a loosely coupled structure. The modules provide easily reusable functions with (where possible) simple implementations, and hooks are provided within the code to enable extensibility. Optimized and competitive performance is, as much as possible, separated out into a small number of lower level components that do not need to be touched by the user. We elaborate on these design choices below.

A. Reusable functions for individual suboperations

It is becoming common practice to provide a Python scripting interface for input and simulation control. However, PySCF goes beyond this by providing a rich set of Python APIs (Application Programming Interfaces) not only for the simulation models but also for many of the individual sub-operations that compose the algorithms. For example, after input parsing, a mean-field Hartree-Fock (HF) or density functional theory (DFT) algorithm comprises many steps, including integral generation, guess initialization, assembling components of the Fock matrix and diagonalizing, and accelerating iterations to self-consistent convergence. All of these suboperations are exposed as PySCF APIs, enabling one to rebuild or modify the self-consistent algorithm at will. Similarly, APIs are exposed for other essential components of electronic structure algorithms, such as integral transformations, density fitting, Hamiltonian manipulation, various many-electron and Green's functions solvers, computation of derivatives, relativistic corrections, and so forth, in essence across all the functionality of PySCF. The package provides a large number of examples to demonstrate how these APIs can be used in customized calculations or methodology development.

With at most some simple initialization statements, the PySCF APIs can be executed at any place and in any order within a code without side-effects. This means that when implementing or extending the code, the user does not need to retain information on the program state and can focus on the physical theory of interest. For instance, using the above example, one can call the function to build a Fock matrix from a given density matrix anywhere in the code, regardless of whether the density matrix in question is related to a larger simulation. From a programming design perspective, this is because within PySCF, no implicit global variables are used and

functions are implemented free of side effects (or with minimal side effects) in a largely functional programming style. The PySCF function APIs generally follow the `NUMPY/SCIPY` API style. In this convention, the input arguments are simple Python built-in datatypes or `NUMPY` arrays, avoiding the need to understand complex objects and structures.

B. Simple implementations

Python is among the simplest of the widely used programming languages and is the main implementation language in PySCF. Apart from a few performance critical functions, over 90% of PySCF is written in Python, with dependencies on only a small number of common external Python libraries (`NUMPY`, `SCIPY`, and `H5PY`).

Implementation language does not hide organizational complexity, however, and structural simplicity in PySCF is achieved via additional design choices. In particular, PySCF uses a mixed object oriented/functional paradigm: complex simulation data (e.g., data on the molecular geometry or cell parameters) and simulation models (e.g., whether a mean-field calculation is a HF or DFT one) are organized in an object oriented style, while individual function implementations follow a functional programming paradigm. Deep object inheritance is rarely used. Unlike packages where external input configuration files are used to control a simulation, the simulation parameters are simply held in the member variables of the simulation model object.

Where possible, PySCF provides multiple implementations of the same algorithm with the same API: one is designed to be easy to read and simple to modify, and another is for optimized performance. For example, the full configuration interaction module contains both a slower but simpler implementation and heavily optimized implementations, specialized for specific Hamiltonian symmetries and spin types. The optimized algorithms have components that are written in C. This dual level of implementation mimics the Python convention of having modules in both pure Python and C with the same API (such as the `PROFILE` and `CPROFILE` modules of the Python standard library). It also reflects the PySCF development cycle, where often a simple reference Python implementation is first produced before being further optimized.

C. Easily modified runtime functionality

In customized simulations, it is often necessary to modify the underlying functionality of a package. This can be complicated in a compiled program due to the need to consider detailed types and compilation dependencies across modules. In contrast, many parts of PySCF are easy to modify due to the design of PySCF as well as the dynamic runtime resolution of methods and “duck typing” of Python. Generally speaking, one can modify the functionality in one part of the code without needing to worry about breaking other parts of the package. For example, one can modify the HF module with a custom Hamiltonian without considering whether it will work in a DFT calculation; the program will continue to run so long as the computational task involves HF and post-HF methods. Furthermore, Python “monkey patching” (replacing functionality at runtime) means that core PySCF routines can be overwritten without even touching the code base of the library.

D. Competitive performance

In many simulations, performance is still the critical consideration. This is typically the reason for implementing code in compiled languages such as FORTRAN or C/C++. In PySCF, the performance gap between Python and compiled languages is partly removed by a heavy reliance on NUMPY and SCIPY, which provide Python APIs to optimized algorithms written in compiled languages. Additional optimization is achieved in PySCF with custom C implementations where necessary. Performance critical spots, which occur primarily in the integral and tensor operations, are implemented in C and heavily optimized. The use of additional C libraries also allows us to achieve thread-level parallelism via OpenMP, bypassing Python's intrinsic multithreading limitations. Since a simulation can often spend over 99% of its runtime in the C libraries, the overhead due to the remaining Python code is negligible. The combination of Python with C libraries ensures that PySCF achieves leading performance in many simulations.

III. A COMMON FRAMEWORK FOR MOLECULES AND CRYSTALLINE MATERIALS

Electronic structure packages typically focus on either molecular or materials simulations and are thus built around numerical approximations adapted to either case. A central goal of PySCF is to enable molecules and materials to be simulated with common numerical approximations and theoretical models. Originally, PySCF started as a Gaussian atomic orbital (AO) molecular code and was subsequently extended to enable simulations in a crystalline Gaussian basis. Much of the seemingly new functionality required in a crystalline materials simulation is, in fact, analogous to the functionality in a molecular implementation, such as

1. Using a Bloch basis. In PySCF, we use a crystalline Gaussian AO basis, which is analogous to a symmetry adapted molecular AO basis.
2. Exploiting translational symmetry by enforcing momentum conservation. This is analogous to handling molecular point group symmetries.
3. Handling complex numbers, given that the matrix elements between Bloch functions are generally complex. This is analogous to the requirements of a molecular calculation with complex orbitals.

Other modifications are unique to the crystalline material setting, including

1. Techniques to handle divergences associated with the long-ranged nature of the Coulomb interaction, since the classical electron–electron, electron–nuclear, and nuclear–nuclear interactions are separately divergent. In PySCF, this is handled via the density fitting integral routines (see below) and by evaluating certain contributions using Ewald summation techniques.
2. Numerical techniques special to periodic functions, such as the fast Fourier transform (FFT), as well as approximations tailored to plane-wave implementations, such as certain pseudopotentials. PySCF supports mixed crystalline Gaussian and plane-wave expressions using analytic integrals as well as FFT on grids.

3. Techniques to accelerate convergence to the thermodynamic limit. In PySCF, such corrections are implemented at the mean-field level by modifying the treatment of the exchange energy, which is the leading finite-size correction.
4. Additional crystal lattice symmetries. Currently, PySCF contains only experimental support for additional lattice symmetries.

In PySCF, we identify the three-index density fitted integrals as the central computational intermediate that allows us to largely unify molecular and crystalline implementations. This is because

1. three-center “density-fitted” Gaussian integrals are key to fast implementations;
2. the use of the FFT to evaluate the potential of a pair density of AO functions, which is needed in fast DFT implementations with pseudopotentials,²⁶ is formally equivalent to density fitting with plane-waves;
3. the density fitted integrals can be adjusted to remove the Coulomb divergences in materials;²⁷ and
4. three-index Coulomb intermediates are sufficiently compact that they can be computed even in the crystalline setting.

PySCF provides a unified density fitting API for both molecules and crystalline materials. In molecules, the auxiliary basis is assumed to be Gaussian AOs, while in the periodic setting, different types of auxiliary bases are provided, including plane-wave functions [in the FFTDF (Fast Fourier Transform Density Fitting) module], crystalline Gaussian AOs [in the GDF (Gaussian Density Fitting) module], and mixed plane-wave-Gaussian functions [in the MDF (Gaussian and Planewave Mixed Density Fitting) module].²⁸ Different auxiliary bases are provided in periodic calculations as they are suited to different AO basis sets: FFTDF is efficient for smooth AO functions when used with pseudopotentials, GDF is more efficient for compact AO functions, and MDF allows a high accuracy treatment of the Coulomb problem, regardless of the compactness of the underlying atomic orbital basis.

Using the above ideas, the general program structure, implementation, and simulation workflow for molecular and materials calculations become very similar. Figure 1 shows an example of the computational workflow adopted in PySCF for performing molecular and periodic post-HF calculations. The same driver functions can be used to carry out generic operations such as solving the HF equations or coupled cluster amplitude equations. However, the implementations of methods for molecular and crystalline systems bifurcate when evaluating k -point dependent quantities, such as the three-center density-fitted integrals, Hamiltonians, and wavefunctions. Nevertheless, if only a single k -point is considered (and especially at the Γ point where all integrals are real), most molecular modules can be used to perform calculations in crystals without modification (see Sec. V).

IV. DEVELOPING WITH PySCF: CASE STUDIES

In this section, we walk through some case studies that illustrate how the functionality of PySCF can be modified and extended. We focus on cases that might be encountered by the average user who does not want to modify the source code but wishes to assemble different existing PySCF APIs to implement a new functionality.

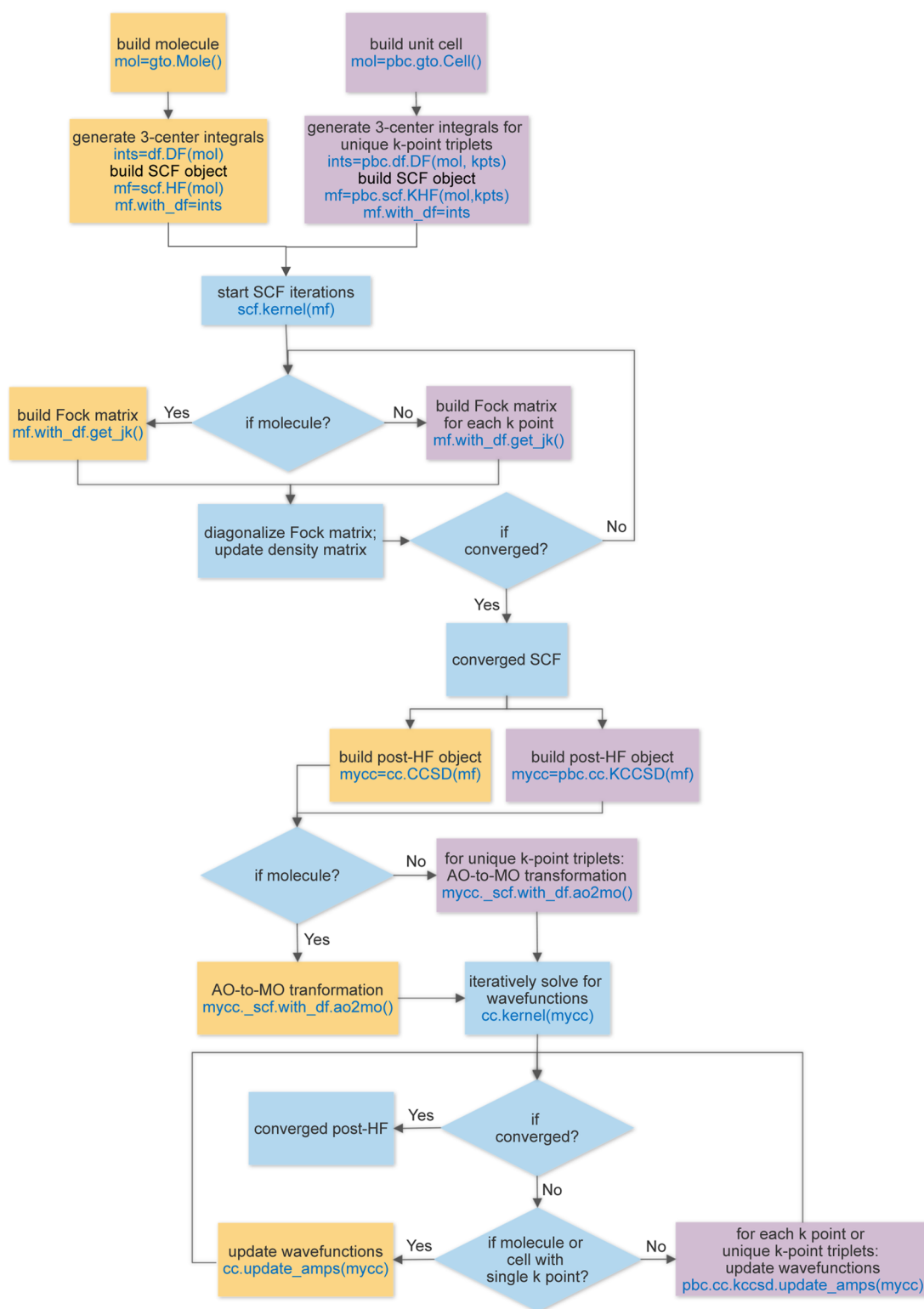


FIG. 1. Illustration of the program workflow for molecular and periodic calculations. The orange and purple boxes indicate functions that are k -point independent and k -point dependent, respectively; the blue boxes indicate generic driver functions that can be used in both molecular and periodic calculations.

A. Case study: Modifying the Hamiltonian

In PySCF, simulation models (i.e., different wavefunction approximations) are always implemented such that they can be used

independently of any specific Hamiltonian with up to two-body interactions. Consequently, the Hamiltonian under study can be easily customized by the user, which is useful for studying model problems or, for example, when trying to interface to different

```
import numpy
import pyscf
mol = pyscf.M()
n = 10
mol.nelectron = n

# Define model Hamiltonian: tight binding on a ring
h1 = numpy.zeros((n, n))
for i in range(n-1):
    h1[i, i+1] = h1[i+1, i] = -1.
h1[n-1, 0] = h1[0, n-1] = -1.

# Build the 2-electron interaction tensor starting from a random 3-index tensor.
tensor = numpy.random.rand(2, n, n)
tensor = tensor + tensor.transpose(0, 2, 1)
eri = numpy.einsum('xpq,xrs->pqrs', tensor, tensor)

# SCF for the custom Hamiltonian
mf = mol.HF()
mf.get_hcore = lambda *args: h1
mf.get_ovlp = lambda *args: numpy.eye(n)

# Option 1: overwrite the attribute mf._eri for the 2-electron interactions
mf._eri = eri
mf.run()

# Option 2: introduce the 2-electron interaction through the Cholesky decomposed tensor.
dfmf = mf.density_fit()
dfmf.with_df._cderi = tensor
dfmf.run()

# Option 3: define a custom HF potential method
def get_veff(mol, dm):
    J = numpy.einsum('xpq,xrs,pq->rs', tensor, tensor, dm)
    K = numpy.einsum('xpq,xrs,qr->ps', tensor, tensor, dm)
    return J - K * .5
mf.get_veff = get_veff
mf.run()

# Call the second order SCF solver in case converging the DIIS-driven HF method
# without a proper initial guess is difficult.
mf = mf.newton().run()

# Run post-HF methods based on the custom SCF object
mf.MP2().run()
mf.CISD().run()
mf.CCSD().run()
mf.CASSCF(4, 4).run()
mf.CASCI(4, 4).run().NEVPT2().run()
mf.TDHF().run()
mf.CCSD().run().EOMIP().run()
mc = shci.SHCISCF(mf, 4, 4).run()
mc = dmrscf.DMRGSCF(mf, 4, 4).run()
```

FIG. 2. Hamiltonian customization and post-HF methods for customized Hamiltonians.

numerical basis approximations. Figure 2 shows several different ways to define one-electron and two-electron interactions in the Hamiltonian followed by subsequent ground and excited state calculations with the custom Hamiltonian. Note that if a method is not compatible with or well defined using the customized interactions, for instance, in the case of solvation corrections, PySCF will raise a Python runtime error in the place where the requisite operations are ill-defined.

B. Case study: Optimizing orbitals of arbitrary methods

The PySCF MCSCF module provides a general purpose quasi-second order orbital optimization algorithm within orbital subspaces (e.g., active spaces) as well as over the complete orbital space. In particular, it is not limited to the built-in CASCI, CASSCF, and multi-reference correlation solvers but allows orbital optimization of any method that provides energies and one- and two-particle

density matrices. For this reason, PySCF is often used to carry out active space orbital optimization for DMRG (density matrix renormalization group), selected configuration interaction, and full configuration interaction quantum Monte Carlo wavefunctions via its native interfaces to Block²⁹ (DMRG), CheMPS2³⁰ (DMRG), Dice^{31–33} (selected CI), Arrow^{31,33,34} (selected CI), and NECI³⁵ [FCIQMC (full configuration interaction quantum Monte Carlo)].

In addition, it is easy for the user to use the MCSCF module to optimize orbitals in electronic structure methods for which the orbital optimization API is not natively implemented. For example, although orbital-optimized MP2³⁶ is not explicitly provided in PySCF, a simple version of it can easily be performed using a short script, as shown in Fig. 3. Without any modifications, the orbital optimization will use a quasi-second order algorithm. We see that the user only needs to write a simple wrapper to provide two functions, namely, `make_rdm12`, which computes the one- and two-particle density matrices, and `kernel`, which computes the total energy.

```
import numpy
import pyscf
class MP2AsFCISolver(object):
    def kernel(self, h1, h2, norb, nelec, ci0=None, ecore=0, **kwargs):
        # Kernel takes the set of integrals from the current set of orbitals
        fakemol = pyscf.M(verbose=0)
        fakemol.nelectron = sum(nelec)
        fake_hf = fakemol.RHF()
        fake_hf._eri = h2
        fake_hf.get_hcore = lambda *args: h1
        fake_hf.get_ovlp = lambda *args: numpy.eye(norb)

        # Build an SCF object fake_hf without SCF iterations to perform MP2
        fake_hf.mo_coeff = numpy.eye(norb)
        fake_hf.mo_occ = numpy.zeros(norb)
        fake_hf.mo_occ[:fakemol.nelectron//2] = 2
        self.mp2 = fake_hf.MP2().run()
        return self.mp2.e_tot + ecore, self.mp2.t2

    def make_rdm12(self, t2, norb, nelec):
        dm1 = self.mp2.make_rdm1(t2)
        dm2 = self.mp2.make_rdm2(t2)
        return dm1, dm2

mol = pyscf.M(atom='H 0 0 0; F 0 0 1.1', basis='ccpvdz')
mf = mol.RHF().run()
# Put in the active space all orbitals of the system
mc = pyscf.mcscf.CASSCF(mf, mol.nao, mol.nelectron)
mc.fcisolver = MP2AsFCISolver()
# Internal rotation inside the active space needs to be enabled
mc.internal_rotation = True
mc.kernel()
```

FIG. 3. Using the general CASSCF solver to implement an orbital-optimized MP2 method.

C. Case study: Implementing an embedding model

As a more advanced example of customization using PySCF, we now illustrate how a simple script with standard APIs enables PySCF to carry out geometry optimization for a wavefunction in Hartree–Fock (WFT-in-HF) embedding model, as shown in Fig. 4 with a configuration interaction with single and double excitations (CISD) solver. Given the Hamiltonian of a system, expressed in terms of the Hamiltonians of a fragment and its environment,

$$H_{\text{sys}} = H_{\text{frag}} + H_{\text{env}} + V_{\text{ee,frag-env}},$$

$$H_{\text{frag}} = h_{\text{core,frag}} + V_{\text{ee,frag}},$$

$$H_{\text{env}} = h_{\text{core,env}} + V_{\text{ee,env}},$$

we define an embedding Hamiltonian for the fragment in the presence of the atoms in the environment as

$$H_{\text{emb}} = h_{\text{eff,frag}} + V_{\text{ee,frag}},$$

$$h_{\text{eff,frag}} = h_{\text{core,frag}} + (h_{\text{core,env}} + V_{\text{eff}}[\rho_{\text{env}}]),$$

$$V_{\text{eff}}[\rho_{\text{env}}] = \int V_{\text{ee,env}}\rho_{\text{env}}(\mathbf{r})d\mathbf{r} + \int V_{\text{ee,frag-env}}\rho_{\text{env}}(\mathbf{r})d\mathbf{r}.$$

Geometry optimization can then be carried out with the approximate nuclear gradients of the embedding problem,

```
import pyscf
frag = pyscf.M(atom='frag.xyz', basis='ccpvtz')
env = pyscf.M(atom='env.xyz', basis='sto-3g')
sys = frag + env

def embedding_gradients(sys):
    # Regular HF energy and nuclear gradients of the entire system
    sys_hf = sys.HF().run()
    grad_sys = sys_hf.Gradients().kernel()

    # Construct a CASCI-like effective 1-electron Hamiltonian for the fragment
    # with the presence of outlying atoms in the environment. dm_env is the
    # density matrix in the environment block
    dm_env = sys_hf.make_rdm1()
    dm_env[frag.nao:,:] = dm_env[:,frag.nao:] = 0
    frag_hcore_eff = (sys_hf.get_hcore() + sys_hf.get_veff(sys, dm_env))[frag.nao:, :frag.nao]

    # Customize the zeroth order calculation by overwriting the core Hamiltonian.
    # HF and CISD now provide the embedding wavefunction on fragment.
    geom_frag = sys.atom_coords(unit='Angstrom')[frag.natm:]
    frag.set_geom_(geom_frag)
    frag_hf = frag.HF()
    frag_hf.get_hcore = lambda *args: frag_hcore_eff
    frag_hf.run()
    frag_ci = frag_hf.CISD().run()

    # The .Gradients() method enables a regular analytical nuclear gradient object
    # to evaluate the Hellmann-Feynman forces on fragment using the first order
    # derivatives of the original fragment Hamiltonian and the variational
    # embedding wavefunction.
    grad_hf_frag = frag_hf.Gradients().kernel()
    grad_ci_frag = frag_ci.Gradients().kernel()

    # Approximate the energy and gradients of the entire system with the post-HF
    # correction on fragment
    approx_e = sys_hf.e_tot + frag_ci.e_tot - frag_hf.e_tot
    approx_grad = grad_sys
    approx_grad[frag.natm:] += grad_ci - grad_hf
    print('Approximate gradients:\n', approx_grad)
    return approx_e, approx_grad

new_sys = pyscf.geomopt.as_pyscf_method(sys, \
    embedding_gradients).Gradients().optimizer().kernel()
```

FIG. 4. An advanced example that implements geometry optimization based on a WFT-in-HF embedding model using standard PySCF APIs.

$$\begin{aligned}
\text{Gradients} &= \left\langle \Psi_{\text{CI}} \left| \frac{\partial H_{\text{sys}}}{\partial X} \right| \Psi_{\text{CI}} \right\rangle \approx \left\langle \Psi_{\text{CI}} \left| \frac{\partial H_{\text{frag}}}{\partial X} \right| \Psi_{\text{CI}} \right\rangle \\
&\quad + \left\langle \Psi_{\text{HF}} \left| \frac{\partial (H_{\text{env}} + V_{\text{ee, frag-env}})}{\partial X} \right| \Psi_{\text{HF}} \right\rangle \\
&= \left\langle \Psi_{\text{CI}} \left| \frac{\partial H_{\text{frag}}}{\partial X} \right| \Psi_{\text{CI}} \right\rangle - \left\langle \Psi_{\text{HF}} \left| \frac{\partial H_{\text{frag}}}{\partial X} \right| \Psi_{\text{HF}} \right\rangle \\
&\quad + \left\langle \Psi_{\text{HF}} \left| \frac{\partial H_{\text{sys}}}{\partial X} \right| \Psi_{\text{HF}} \right\rangle \\
&\approx \left\langle \Psi_{\text{frag,CI}} \left| \frac{\partial H_{\text{frag}}}{\partial X} \right| \Psi_{\text{frag,CI}} \right\rangle - \left\langle \Psi_{\text{frag,HF}} \left| \frac{\partial H_{\text{frag}}}{\partial X} \right| \Psi_{\text{frag,HF}} \right\rangle \\
&\quad + \left\langle \Psi_{\text{HF}} \left| \frac{\partial H_{\text{sys}}}{\partial X} \right| \Psi_{\text{HF}} \right\rangle,
\end{aligned}$$

where the fragment wavefunctions $\Psi_{\text{frag,HF}}$ and $\Psi_{\text{frag,CI}}$ are obtained from the embedding Hamiltonian H_{emb} . The code snippet in Fig. 4 demonstrates the kind of rapid prototyping that can be carried out using PySCF APIs. In particular, this demonstration combines the APIs for *ab initio* energy evaluation, analytical nuclear gradient computation, computing the HF potential for an arbitrary density matrix, Hamiltonian customization, and customizing the nuclear gradient solver in a geometry optimization.

V. SUMMARY OF EXISTING METHODS AND RECENT ADDITIONS

In this section, we briefly summarize major current capabilities of the PySCF package. These capabilities are listed in Table I, and the details are presented in Subsections V A–V H.

TABLE I. Major features of PySCF as of version 1.7.1.

Methods	Molecules	Solids	Comments
HF	Yes	Yes	~10 000 AOs ^a
MP2	Yes	Yes	~1 500 MOs ^a
DFT	Yes	Yes	~10 000 AOs ^a
TDDFT/TDHF/TDA/CIS	Yes	Yes	~10 000 AOs ^a
G ₀ W ₀	Yes	Yes	~1 500 MOs ^a
CISD	Yes	Yes ^b	~1 500 MOs ^a
FCI	Yes	Yes ^b	~(18e, 18o) ^a
IP/EA-ADC(2)	Yes	No	~500 MOs ^{a,c}
IP/EA-ADC(2)-X	Yes	No	~500 MOs ^{a,c}
IP/EA-ADC(3)	Yes	No	~500 MOs ^{a,c}
CCSD	Yes	Yes	~1500 MOs ^a
CCSD(T)	Yes	Yes	~1500 MOs ^a
IP/EA/EE-EOM-CCSD ^d	Yes	Yes	~1500 MOs ^a
MCSCF	Yes	Yes ^b	~3000 AOs, ^a 30–50 active orbitals ^e
MRPT	Yes	Yes ^b	~1500 MOs, ^a 30–50 active orbitals ^e
QM/MM	Yes	No	
Semiempirical	Yes	No	MINDO3
Relativity	Yes	No	ECP and scalar-relativistic corrections for all methods; two-component methods for HF, DFT, DMRG, and SHCI; four-component methods for HF and DFT
Gradients	Yes	No	HF, MP2, DFT, TDDFT, CISD, CCSD, CCSD(T), MCSCF, and MINDO3
Hessian	Yes	No	HF and DFT
Orbital localizer	Yes	Yes	NAO, meta-Löwdin, IAO/IBO, VVO/LIVVO, Foster–Boys, Edmiston–Ruedenberg, Pipek–Mezey, and maximally localized Wannier functions
Properties	Yes	Yes ^f	EFGs, Mössbauer spectroscopy, NMR, magnetizability, and polarizability
Solvation	Yes	No	ddCOSMO, ddPCM, and polarizable embedding
AO, MO integrals	Yes	Yes	One-electron and two-electron integrals
Density fitting	Yes	Yes	HF, DFT, MP2, and CCSD
Symmetry	Yes	No ^g	D _{2h} and subgroups for molecular HF, MCSCF, and FCI

^aAn estimate based on a single SMP node with 128 GB memory without density fitting.

^bΓ-point only.

^cIn-core implementation limited by storing two-electron integrals in memory.

^dPerturbative corrections to IP and EA via IP-EOM-CCSD* and EA-EOM-CCSD* are available for both molecules and crystals.

^eUsing an external DMRG, SHCI, or FCIQMC program (as listed in Sec. IV B) as the active space solver.

^fEFGs and Mössbauer spectra only.

^gExperimental support for point-group and time-reversal symmetries in crystals at the SCF and MP2 levels.

```
import pyscf
mol = pyscf.M(atom = 'N O O O; N O O 1.1', basis = 'ccpvdz')
mf = mol.RKS()
mf.xc = 'CAMB3LYP'
mf.xc = '''0.19*SR_HF(0.33) + 0.65*LR_HF(0.33) + 0.46*ITYH + 0.35*B88, 0.19*VWN5 + 0.81*LYP'''
mf.xc = 'RSH(0.33, 0.65, -0.46) + 0.46*ITYH + 0.35*B88, 0.19*VWN5 + 0.81*LYP'
e_mf = mf.kernel()
```

FIG. 5. An example of two customized RSH functionals that are equivalent to the CAM-B3LYP functional.

A. Hartree-Fock and density functional theory methods

The starting point for many electronic structure simulations is a self-consistent field (SCF) calculation. PySCF implements Hartree-Fock (HF) and density functional theory (DFT) with a variety of Slater determinant references, including restricted closed-shell, restricted open-shell, unrestricted, and generalized (noncollinear spin) references,^{37,38} for both molecular and crystalline (k -point) calculations. Through an interface to the LIBXC³⁹ and XCFUN⁴⁰ libraries, PySCF also supports a wide range of predefined exchange-correlation (XC) functionals, including the local density approximation (LDA), generalized gradient approximations (GGAs), hybrids, meta-GGAs, nonlocal correlation functionals (VV10⁴¹), and range-separated hybrid (RSH) functionals. In addition to the predefined XC functionals, the user can also create customized functionals in a DFT calculation, as shown in Fig. 5.

Because PySCF uses a Gaussian AO representation, the SCF computation is usually dominated by Gaussian integral evaluation. Through the efficient Gaussian integral engine LIBCINT,⁴² the molecular SCF module can be used with more than 10 000 basis functions on a symmetric multiprocessing (SMP) machine, without resorting to any integral approximations such as screening. Further speed-up

can be achieved through Gaussian density fitting, and the pseudo-spectral approach (SGX) is implemented to speed up the evaluation of exchange in large systems.^{43–45}

In crystalline systems, HF and DFT calculations can be carried out either at a single point in the Brillouin zone or with a k -point mesh. The cost of the crystalline SCF calculation depends on the nature of the crystalline Gaussian basis and the associated density fitting. PySCF supports Goedecker-Teter-Hutter (GTH) pseudopotentials⁴⁶ that can be used with the associated basis sets (developed by the CP2K group).^{26,47} Pseudopotential DFT calculations are typically most efficiently done using plane-wave density fitting (FFTDF). Alternatively, all-electron calculations can be performed with standard basis sets, and the presence of sharp densities means that Gaussian density fitting performs better. Gaussian density fitting is also the algorithm of choice for calculations with HF exchange. Figure 6 shows an example of the silicon band structures computed using a GTH-LDA pseudopotential with FFTDF and in an all-electron calculation using GDF.

B. Many-body methods

Starting from a SCF HF or DFT wavefunction, various many-body methods are available in PySCF, including Møller-Plesset second-order perturbation theory (MP2), multi-reference perturbation theory (MRPT),^{48,49} configuration interaction (CI),^{50–53} coupled cluster (CC),^{54–63} multi-configuration self-consistent field (MCSCF),^{64,65} algebraic diagrammatic construction (ADC),^{66–70} and G_0W_0 ^{71–74} methods. The majority of these capabilities are available for both molecules and crystalline materials.

1. Molecular implementations

The PySCF CI module implements solvers for configuration interaction with single and double excitations (CISD) and a general full configuration interaction (FCI) solver that can treat fermion, boson, and coupled fermion-boson Hamiltonians. The FCI solver is heavily optimized for its multithreaded performance and can efficiently handle active spaces with up to 18 electrons in 18 orbitals.

The CC module implements coupled cluster theory with single and double excitations (CCSD)^{56,61} and with the perturbative triples correction [CCSD(T)].⁵⁷ Λ -equation solvers are implemented to compute one- and two-particle density matrices as well as the analytic nuclear gradients for the CCSD and CCSD(T) methods.^{55,58,59} PySCF also implements various flavors of equation-of-motion CCSD to compute electron affinities (EAs), ionization

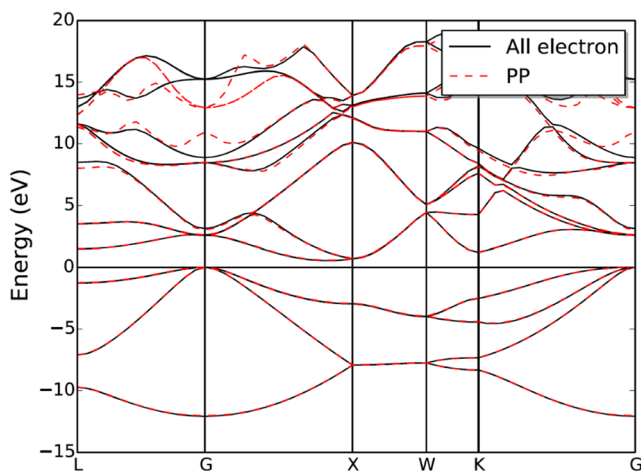


FIG. 6. All-electron and pseudopotential LDA band structures of the Si crystal. Reprinted with permission from, Sun *et al.*, J. Chem. Phys. **147**, 164119 (2017). Copyright 2017 AIP Publishing LLC.

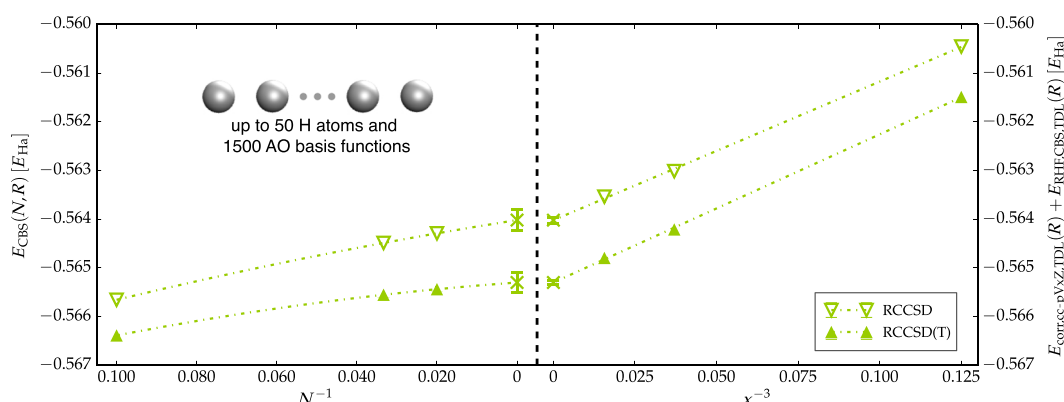


FIG. 7. Energies of a hydrogen chain computed at the restricted CCSD and CCSD(T) levels extrapolated to the complete basis set (CBS) and thermodynamic limits. The left-hand panel shows extrapolation of $E_{\text{CBS}}(N)$ vs $1/N$, where N is the number of atoms, while the right-hand panel shows extrapolation of $E_{\text{CBS}}(N)$ vs $1/x^3$ with x equal to 2, 3, and 4 corresponding to double-, triple-, and quadruple-zeta basis, respectively. Adapted from Ref. 77.

potentials (IPs), neutral excitation energies (EEs), and spin-flip (SF) excitation energies.^{54,60,62,63} Experimental support for beyond double corrections to IP and EA via IP-EOM-CCSD^{75,76} and EA-EOM-CCSD^{*} is also available. For very large basis sets, PySCF provides an efficient AO-driven pathway that allows calculations with more than 1500 basis functions. An example of this is shown in Fig. 7, where the largest CCSD(T) calculation contains 50 electrons and 1500 basis functions.⁷⁷

Second- and third-order algebraic diagrammatic construction (ADC) methods are also available in PySCF for the calculation of molecular electron affinities and ionization potentials^{66–70} [EA/IP-ADC(n), $n = 2, 3$]. These have a lower cost than EA/IP-EOM-CCSD. The advantage of the ADC methods over EOM-CCSD is that their amplitude equations can be solved in one iteration and the eigenvalue problem is Hermitian, which lowers the cost of computing the EA/IP energies and transition intensities.

The MCSCF module provides complete active space configuration interaction (CASSCF) and complete active space self-consistent field (CASSCF) methods for multi-reference problems. As discussed in Sec. IV B, the module also provides a general second-order orbital optimizer⁷⁸ that can optimize the orbitals of external methods, with native interfaces for the orbital optimization of the density matrix renormalization group (DMRG),^{29,30} full configuration interaction quantum Monte Carlo (FCIQMC),^{35,79} and selected configuration interaction wavefunctions.^{31,32} Starting from a CASSCF or CASSCF wavefunction, PySCF also implements the strongly contracted second-order n -electron valence perturbation theory (SC-NEVPT2) in the MRPT module to include additional dynamic correlation. Together with external active-space solvers, this enables one to treat relatively large active spaces for such calculations, as illustrated in Fig. 8.

2. Crystalline implementations

As discussed in Sec. III, the PySCF implementations of many-body methods for crystalline systems closely parallel their molecular implementations. In fact, all molecular modules can be used to carry out calculations in solids at the Γ -point, and many modules (those

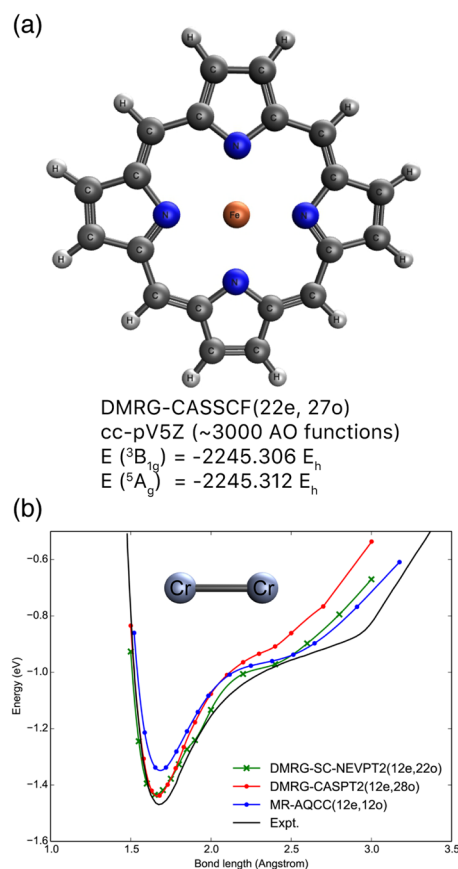


FIG. 8. (a) Ground-state energy calculations for Fe(II)-porphine at the DMRG-CASSCF/cc-pV5Z level with an active space of 22 electrons in 27 orbitals.⁷⁸ (b) Potential energy curve for Cr₂ at the DMRG-SC-NEVPT2 (12e, 22o) level, compared to the results from other methods. Adapted with permission from Guo *et al.*, J. Chem. Theory Comput. **12**, 1583 (2016). Copyright (2016) American Chemical Society.

```
import pyscf
cell = pyscf.M(atom=..., a=...) # 'a' defines lattice vectors
mf = cell.HF(kpt = [0.23,0.23,0.23]).run()
# Use PBC CCSD class so integrals are handled
# correctly with respect to Coulomb divergences
mycc = pyscf.pbc.cc.CCSD(mf)
# molecular CCSD code used to compute correlation energy at single k-point
converged, ecorr = pyscf.cc.ccsd.kernel(mycc)
```

FIG. 9. Illustration of using the molecular code to compute an energy in the crystal at a single k -point.

supporting complex integrals) can be used at any other single k -point. Such single k -point calculations only require the appropriate periodic integrals to be supplied to the many-body solver (Fig. 9). For those modules that support complex integrals, twist averaging can then be performed to sample the Brillouin zone. To use savings from k -point symmetries, an additional summation over momentum conserving k -point contributions needs to be explicitly implemented. Such implementations are provided for MP2, CCSD, CCSD(T), IP/EA-EOM-CCSD²⁷ and EE-EOM-CCSD,⁸⁰ and G_0W_0 . For example, Fig. 10 shows the MP2 correlation energy and the CIS excitation energy of MgO, calculated using periodic

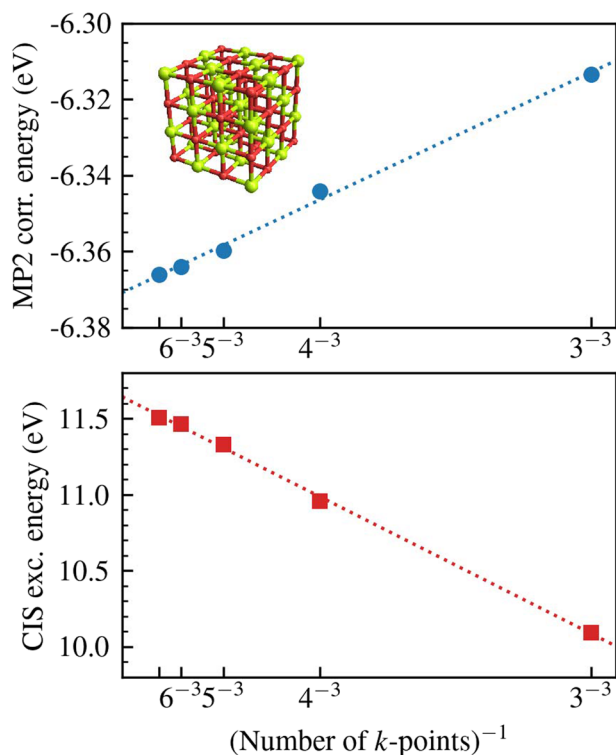


FIG. 10. Periodic MP2 correlation energy per unit cell (top) and CIS excitation energy (bottom) as a function of the number of k -points sampled in the Brillouin zone for the MgO crystal.

density-fitted implementations; the largest system shown, with a $7 \times 7 \times 7$ k -point mesh, correlates 5488 valence electrons in 9261 orbitals. Furthermore, Fig. 11 shows some examples of periodic correlated calculations on NiO carried out using the G_0W_0 and CCSD methods.

C. Efficiency

In Table I, we provide rough estimates of the sizes of problems that can be tackled using PySCF for each electronic structure method. Figures 7, 8, 10, and 11 illustrate some real-world examples of calculations performed using PySCF. Note that the size of system that can be treated is a function of the computational resources available; the estimates given above assume relatively standard and modest computational resources, e.g., a node of a cluster, or a few dozen cores. For more details of the runtime environment and program settings for similar performance benchmarks, we refer readers to the benchmark page of the PySCF website (www.PySCF.org). The implementation and performance of PySCF on massively parallel architectures is discussed in Sec. V H.

For molecular calculations using mean-field methods, PySCF can treat systems with more than 10 000 AO basis functions without difficulty. Figure 15 shows the time of building the Fock matrix for a large water cluster with more than 12 000 basis functions. With the integral screening threshold set to 10^{-13} a.u., it takes only around 7 h on one computer node with 32 CPU cores. Applying MPI (Message Passing Interface) parallelization further reduces the Fock-build time (see Sec. V H). For periodic boundary calculations at the DFT level using pure XC functionals, even larger systems can be treated using pseudopotentials and a multi-grid implementation. Table II presents an example of such a calculation, where for the largest system considered ($[\text{H}_2\text{O}]_{512}$ with more than 25 000 basis functions), the Fock-build time is about an hour or less on a single node.

To demonstrate the efficiency of the many-body method implementations, in Tables III and IV, we show timing data of exemplary CCSD and FCI calculations. It is clear that systems with more than 1500 basis functions can be easily treated at the CCSD level and that the FCI implementation in PySCF is very efficient. In a similar way, the estimated performance for other many-body methods implemented in PySCF is listed in Table I.

D. Properties

At the mean-field level, the current PySCF program can compute various nonrelativistic and four-component relativistic

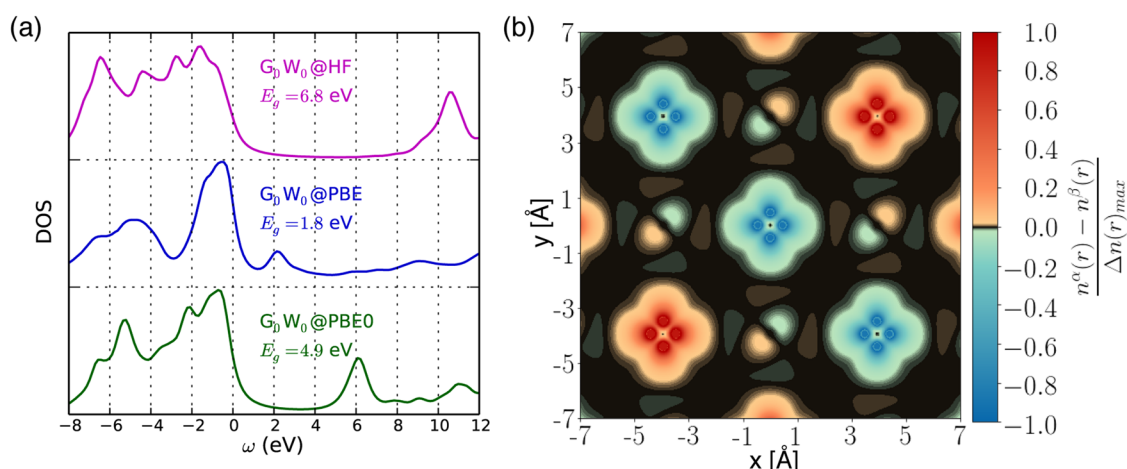


FIG. 11. Electronic structure calculations for antiferromagnetic NiO. (a) Density of states and bandgaps computed by G_0W_0 . (b) Normalized spin density on the (100) surface by CCSD (the Ni atom is located at the center). Adapted from Ref. 81.

molecular properties. These include NMR shielding and spin-spin coupling tensors,^{84–89} electronic g-tensors,^{90–93} nuclear spin-rotation constants and rotational g-tensors,^{94,95} hyperfine coupling (HFC) tensors,^{96,97} electron spin-rotation (ESR) tensors,^{98,99}

TABLE II. Wall time (in seconds) for building the Fock matrix in a supercell DFT calculation of water clusters with the GTH-TZVP26,47 basis set using the SVWN⁸² and the PBE⁸³ XC functionals and the corresponding pseudopotentials. Integral screening and lattice summation cutoff were controlled by an overall threshold of 10^{-6} a.u. for Fock matrix elements. The calculations were performed on one computer node with 32 Intel Xeon Broadwell (E5-2697v4) processors.

System	N_{AO}^a	SVWN	PBE
[H ₂ O] ₃₂	1280	8	23
[H ₂ O] ₆₄	2560	20	56
[H ₂ O] ₁₂₈	5120	74	253
[H ₂ O] ₂₅₆	12 800	276	1201
[H ₂ O] ₅₁₂	25 600	1279	4823

^aNumber of AO basis functions.

TABLE III. Wall time (in seconds) for the first CCSD iteration in AO-driven CCSD calculations on hydrogen chains. The threshold of integral screening was set to 10^{-13} a.u. For these hydrogen chain molecules, CCSD takes around 10 iterations to converge. The calculations were performed on one computer node with 28 Intel Xeon Broadwell (E5-2697v4) processors.

System/basis set	N_{occ}^a	N_{virt}^b	Time
H ₃₀ /cc-pVQZ	15	884	621
H ₃₀ /cc-pV5Z	15	1631	6887
H ₅₀ /cc-pVQZ	25	1472	8355

^aNumber of active occupied orbitals.

^bNumber of active virtual orbitals.

magnetizability tensors,^{94,100,101} zero-field splitting (ZFS) tensors,^{102–104} as well as static and dynamic polarizability and hyperpolarizability tensors. The contributions from spin-orbit coupling and spin-spin coupling can also be calculated and included in the g-tensors, HFC tensors, ZFS tensors, and ESR tensors. In magnetic property calculations, an approximate gauge-origin invariance is ensured for NMR shielding, g-tensors, and magnetizability tensors via the use of gauge including atomic orbitals.^{94,100,101,105,106}

Electric field gradients (EFGs) and Mössbauer parameters^{107–109} can be computed using either the mean-field electron density or the correlated density obtained from non-relativistic Hamiltonians, spin-free exact-two-component (X2C) relativistic Hamiltonians,^{110–114} or four-component methods in both molecules and crystals.

Finally, analytic nuclear gradients for the molecular ground state are available at the mean-field level and for many of the electron correlation methods such as MP2, CCSD, CISD, CASCI, and CASSCF (see Table I). The CASCI gradient implementation supports the use of external solvers, such as DMRG, and provides gradients for such methods. PySCF also implements the analytical gradients of time-dependent density functional theory (TDDFT) with or without the Tamm-Dancoff approximation (TDA) for excited state geometry optimization. The spin-free X2C relativistic Hamiltonian,

TABLE IV. Wall time (in seconds) for one FCI iteration for different active-space sizes. The calculations were performed on one computer node with 32 Intel Xeon Broadwell (E5-2697v4) processors.

Active space	Time
(12e, 12o)	0.1
(14e, 14o)	0.7
(16e, 16o)	8
(18e, 18o)	156

frozen core approximations, solvent effects, and molecular mechanics (MM) environments can be combined with any of the nuclear gradient methods. Vibrational frequency and thermochemical analysis can also be performed using the analytical Hessians from mean-field level calculations or numerical Hessians of methods based on the numerical differentiation of analytical gradients.

E. Orbital localization

PySCF provides two kinds of orbital localizations in the LO module. The first kind localizes orbitals based on the atomic character of the basis functions and can generate intrinsic atomic orbitals (IAOs),¹¹⁵ natural atomic orbitals (NAOs),¹¹⁶ and meta-Löwdin orbitals.¹¹⁷ These AO-based local orbitals can be used to carry out a reliable population analysis in arbitrary basis sets.

The second kind optimizes a cost function to produce localized orbitals. PySCF implements Boys localization,¹¹⁸ Edmiston–Ruedenberg localization,¹¹⁹ and Pipek–Mezey localization.¹²⁰ Starting from the IAOs, one can also use orbital localization based on the Pipek–Mezey procedure to construct the intrinsic bond orbitals (IBOs).¹¹⁵ A similar method can also be used to construct localized intrinsic valence virtual orbitals that can be used to assign the core-excited states.¹²¹ The optimization in these localization routines takes advantage of the second order co-iterative augmented Hessian (CIAH) algorithm¹²² for rapid convergence.

For crystalline calculations with k -point sampling, PySCF also provides maximally localized Wannier functions (MLWFs) via a native interface to the WANNIER90 program.¹²³ Different types of orbitals are available as initial guesses for the MLWFs, including the atomic orbitals provided by WANNIER90, meta-Löwdin orbitals,¹¹⁷ and localized orbitals from the selected columns of density matrix (SCDM) method.^{124,125} Figure 12 illustrates the IBOs and MLWFs of diamond computed by PySCF.

F. QM/MM and solvent

PySCF incorporates two continuum solvation models, namely, the conductor-like screening model¹²⁶ (COSMO) and the polarizable continuum model using the integral equation formalism^{127,128} (IEF-PCM). Both of them are implemented efficiently via a domain decomposition (dd) approach^{129–133} and are compatible with most of the electronic structure methods in PySCF. Furthermore, besides equilibrium solvation where the solvent polarization is governed by

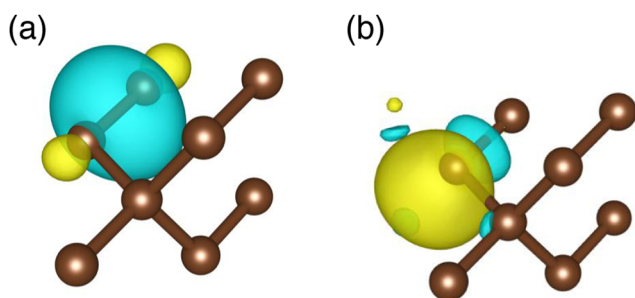


FIG. 12. (a) IBOs for diamond at the Γ -point (showing one σ bond); (b) MLWFs for diamond computed within the valence IAO subspace (showing one sp^3 orbital).

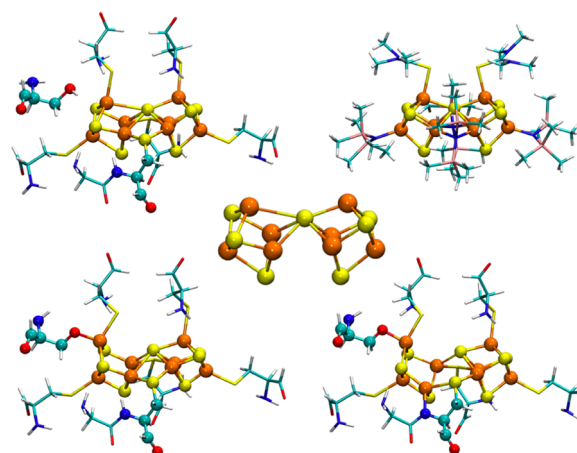


FIG. 13. Illustration of P-cluster (the $[\text{Fe}_8\text{S}_7]$ cluster of nitrogenase) calculations where the COSMO solvation model was used to mimic the protein environment of nitrogenase beyond the first coordination sphere. Fe, orange; S, yellow; C, cyan; O, red; N, blue; H, white; Si, pink. Adapted from Ref. 134.

the static electric susceptibility, non-equilibrium solvation can also be treated within the framework of TDDFT in order to describe fast solvent response with respect to abrupt changes in the solute charge density. As an example, in Ref. 134, the COSMO method was used to mimic the protein environment of nitrogenase in electronic structure calculations for the P-cluster (Fig. 13). For excited states generated by TDA, the polarizable embedding model¹³⁵ can also be used through an interface to the external library CPPE.^{135,136}

Currently, PySCF provides some limited functionality for performing QM/MM calculations by adding classical point charges to the QM region. The implementation supports all molecular electronic structure methods by decorating the underlying SCF methods. In addition, MM charges can be used together with the X2C method and implicit solvent treatments.

G. Relativistic treatments

PySCF provides several ways to include relativistic effects. In the framework of scalar Hamiltonians, spin-free X2C theory,¹³⁷ scalar effective core potentials¹³⁸ (ECPs), and relativistic pseudopotentials can all be used for all methods in calculations of the energy, nuclear gradients, and nuclear Hessians. At the next level of relativistic approximations, PySCF provides spin-orbit ECP integrals and one-body and two-body spin-orbit interactions from the Breit–Pauli Hamiltonian and X2C Hamiltonian for the spin-orbit coupling effects.¹³⁹ Two-component Hamiltonians with the X2C one-electron approximation and four-component Dirac–Coulomb, Dirac–Coulomb–Gaunt, and Dirac–Coulomb–Breit Hamiltonians are all supported in mean-field molecular calculations.

H. MPI implementations

In PySCF, distributed parallelism with MPI is implemented via an extension to the PySCF main library known as MPI4PySCF. The current MPI extension supports the most common methods in quantum chemistry and crystalline material computations. Table V

TABLE V. Methods with MPI support. For solids, MPI support is currently provided only at the level of parallelization over k -points.

Methods	Molecules	Solids
HF	Yes	Yes
DFT	Yes	Yes
MP2	Yes ^a	Yes
CCSD	Yes ^a	Yes

^aClosed shell systems only.

lists the available MPI-parallel alternatives to the default serial (OpenMP) implementations. The MPI-enabled modules implement almost identical APIs to the serial ones, allowing the same script to be used for serial jobs and MPI-parallel jobs (Fig. 14). The efficiency of the MPI implementation is demonstrated in Fig. 15, which shows the wall time and speed-up of Fock builds for a system with 12 288 AOs with up to 64 MPI processes, each with 32 OpenMP threads.

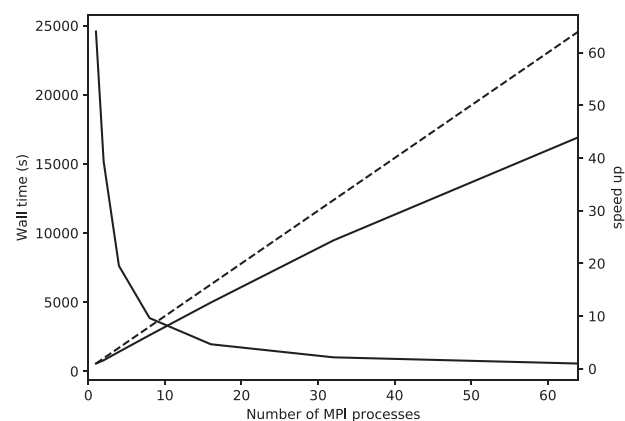
To retain the simplicity of the PySCF package structure, we use a server–client mechanism to execute the MPI parallel code. In particular, we use MPI to start the Python interpreter as a daemon that receives both the functions and data on remote nodes. When a parallel session is activated, the master process sends the functions and data to the daemons. The function object is decoded remotely and then executed. For example, when building the Fock matrix in the PySCF MPI implementation, the Fock-build function running on the master process first sends itself to the Python interpreters running on the clients. After the function is decoded on the clients, input variables (such as the density matrix) are distributed by the master process through MPI. Each client evaluates a subset of the

```
# run in cmdline:
# mpirun -np 4 python input.py

import pyscf
mol = pyscf.M(...)

# Serial task
from pyscf import dft
mf = dft.RKS(mol).run(xc='b3lyp')
J, K = mf.get_jk(mol, mf.make_rdm1())

# MPI-parallel task
from mpi4pyscf import dft
mf = dft.RKS(mol).run(xc='b3lyp')
J, K = mf.get_jk(mol, mf.make_rdm1())
```

FIG. 14. Code snippet showing the similarity between serial and MPI-parallel DFT calculations.**FIG. 15.** Computation wall time of building the Fock matrix for the $[\text{H}_2\text{O}]_{512}$ cluster at the HF/VDZ level (12 288 AO functions) using PySCF's MPI implementation. Each MPI process contains 32 OpenMP threads, and the speed-up is compared to the single-node calculation with 32 OpenMP threads.

four-center two-electron integrals (with load balancing performed among the clients) and constructs a partial Fock matrix, similarly to the Fock-build functions in other MPI implementations. After sending the partial Fock matrices back to the master process, the client suspends itself until it receives the next function. The master process assembles the Fock matrices and then moves on to the next part of the code. The above strategy is quite different from the traditional MPI programs that hard-code the MPI functionality into the code and initiate the MPI parallel context at the beginning of the program. This PySCF design brings the important benefit of being able to switch on and off MPI parallelism freely in the program without the need to be aware of the MPI-parallel context (see Ref. 1 for a more detailed discussion of PySCF MPI mode innovations).

VI. THE PySCF SIMULATION ECOSYSTEM

PySCF is widely used as a development tool, and many groups have developed and made available their own projects that either interface to PySCF or can be used in a tightly coupled manner to access a greater functionality. We provide a few examples of the growing PySCF ecosystem below, which we separate into use cases: (1) external projects to which PySCF provides and maintains a native interface and (2) external projects that build on PySCF.

A. External projects with native interfaces

PySCF currently maintains a few native interfaces to external projects, including

- GEOMETRIC¹⁴⁰ and PYBERNY.¹⁴¹ These two libraries provide the capability to perform geometry optimization, and interfaces to them are provided in the PySCF GEOMOPT module. As shown in Fig. 4, given a method that provides energies and nuclear gradients, the geometry optimization module generates an object that can then be used by these external optimization libraries.

- DFTD3.^{142,143} This interface allows us to add the DFTD3¹⁴² correction to the total ground state energy as well as to the nuclear gradients in geometry optimizations.
- DMRG, SHCI, and FCIQMC programs (BLOCK,²⁹ CHEMPS2,³⁰ DICE,^{31–33} ARROW,^{31,33,34} and NECI³⁵). These interfaces closely follow the conventions of PySCF's FCI module. As such, they can be used to replace the FCI solver in MCSCF methods (CASCI and CASSCF) to study large active space multi-reference problems.
- LIBXC³⁹ and XCFUN.⁴⁰ These two libraries are tightly integrated into the PySCF code. While the PySCF DFT module allows the user to customize exchange–correlation (XC) functionals by linearly combining different functionals, the individual XC functionals and their derivatives are evaluated within these libraries.
- TBLIS.^{144–146} The tensor contraction library TBLIS offers a similar functionality to the `numpy.einsum` function while delivering substantial speed-ups. Unlike the BLAS-based “transpose-GEMM-transpose” scheme that involves a high memory footprint due to the transposed tensor intermediates, TBLIS achieves optimal tensor contraction performance without such memory overhead. The TBLIS interface in PySCF provides an `einsum` function that implements the `numpy.einsum` API but with the TBLIS library as the contraction back-end.
- CPPE.^{135,136} This library provides a polarizable embedding solvent model and can be integrated into PySCF calculations for ground-state mean-field and post-SCF methods. In addition, an interface to TDA is currently supported for excited-state calculations.

B. External projects that build on PySCF

There are many examples in the literature of quantum chemistry and electronic structure simulation packages that build on PySCF. The list below is by no means exhaustive but gives an idea of the range of projects using PySCF today.

1. *Quantum Monte Carlo*. Several quantum Monte Carlo programs, such as QMCPACK,¹⁴⁷ pyQMC,¹⁴⁸ QWALK,¹⁴⁹ and HANDE,¹⁵⁰ support reading wavefunctions and/or Hamiltonians generated by PySCF. In the case of pyQMC, PySCF is integrated as a dependent module.
2. *Quantum embedding packages*. Many flavors of quantum embedding, including density matrix embedding and dynamical mean-field theory, have been implemented on top of PySCF. Examples of such packages include QSoME,^{151–153} pDMET,^{154,155} PyDMFET,¹⁵⁶ POTATO,^{157,158} and openQEMIST,¹⁶ which all use PySCF to manipulate wavefunctions and embedding Hamiltonians and to provide many-electron solvers.
3. *General quantum chemistry*. PySCF can be found as a component of tools developed for many different kinds of calculations, including localized active space self-consistent field (LASSCF),¹⁵⁴ multiconfiguration pair-density functional theory (MC-PDFT),¹⁵⁹ and state-averaged CASSCF energy and analytical gradient evaluation (all these use the PySCF MCSCF module to optimize multi-reference wavefunctions), as well as for localized orbital construction via the PYWANNIER90 library.¹⁵⁵ The PyMBE package,¹⁶⁰ which implements the many-body

expanded full CI method,^{161–164} utilizes PySCF to perform all the underlying electronic structure calculations. Green's function methods such as the second-order Green's function theory (GF2) and the self-consistent GW approximation have been explored using PySCF as the underlying *ab initio* infrastructure.¹⁶⁵ In the linear scaling program LSQC,^{166,167} PySCF is used to generate reference wavefunctions and integrals for the cluster-in-molecule local correlation method. The APDFT (alchemical perturbation density functional theory) program^{168,169} interfaces to PySCF for QM calculations. In the PySCF-NAO project,¹⁷⁰ large-scale ground-state and excited-state methods are implemented based on additional support for numerical atomic orbitals, which has been integrated into an active branch of PySCF. The PyFLOSIC package¹⁷¹ evaluates self-interaction corrections with the Fermi–Löwdin orbitals in conjunction with the PySCF DFT module. Furthermore, PySCF FCI capabilities are used in the MOLSTURM package¹⁷² for the development of Coulomb Sturmian basis functions, and PySCF post-HF methods appear in VELOXCHEM¹⁷³ and ADC¹⁷⁴ for spectroscopic and excited-state simulations.

VII. BEYOND ELECTRONIC STRUCTURE

A. PySCF in the materials genome initiative and machine learning

As discussed in Sec. I, one of our objectives when developing PySCF was to create a tool that could be used by non-specialist researchers in other fields. With the integration of machine learning techniques into molecular and materials simulations, we find that PySCF is being used in many applications in conjunction with machine learning. For example, the flexibility of the PySCF DFT module has allowed it to be used to test exchange–correlation functionals generated by machine-learning protocols in several projects^{7,8} and has been integrated into other machine learning workflows.^{9,10} PySCF can be used as a large-scale computational engine for quantum chemistry data generation.^{5,6} In the context of machine learning of wavefunctions, PySCF has also been used as the starting point to develop neural network based approaches for SCF initial guesses,¹¹ for the learning of HF orbitals by the DeepMind team,¹² and for Hamiltonian integrals used by fermionic neural nets in NETKET.¹³

B. PySCF in quantum information science

Another area where PySCF has been rapidly adopted as a development tool is in the area of quantum information science and quantum computing. This is likely because Python is the *de facto* standard programming language in the quantum computing community. For example, PySCF is one of the standard prerequisites to carry out molecular simulations in the OPENFERMION¹⁴ library, the QISKIT-AQUA¹⁵ library, and the OPENQEMIST¹⁶ package. Through PySCF's GitHub page, we see a rapidly increasing number of quantum information projects that include PySCF as a program dependency.

VIII. OUTLOOK

After five years of development, the PySCF project can probably now be considered to be a feature complete and mature tool.

Although no single package can be optimal for all tasks, we believe that PySCF to a large extent meets its original development criteria of forming a library that is useful not only in simulations but also in enabling the customization and development of new electronic structure methods.

With the recent release of version 1.7, the current year marks the end of development of the version 1 branch of PySCF. As we look toward PySCF version 2, we expect to build additional innovations, for example, in the areas of faster electronic structure methods for very large systems, further support and integration for machine learning and quantum computing applications, better integration of high-performance computing libraries and more parallel implementations, and perhaps even forays into dynamics and classical simulations. Beyond feature development, we will expand our efforts in documentation and in quality assurance and testing. We expect the directions of implementation to continue to be guided by and organically grow out of the established PySCF ecosystem. However, regardless of the scientific directions and methods implemented within PySCF, the guiding philosophy described in this article will continue to lie at the heart of PySCF's development. We believe that these guiding principles will help ensure that PySCF remains a powerful and useful tool in the community for many years to come.

ACKNOWLEDGMENTS

As a large package, the development of PySCF has been supported by different sources. Support from the U.S. National Science Foundation via Award No. 1931258 (T.C.B., G.K.-L.C., and L.K.W.) is acknowledged to integrate high-performance parallel infrastructure and faster mean-field methods into PySCF. Support from the U.S. National Science Foundation via Award No. 1657286 (G.K.-L.C.) and Award No. 1848369 (T.C.B.) is acknowledged for various aspects of the development of many-electron wavefunction methods with periodic boundary conditions. Support for integrating PySCF into quantum computing platforms was provided partially by the Department of Energy via Award No. 19374 (G.K.-L.C.). The Simons Foundation is gratefully acknowledged for providing additional support for the continued maintenance and development of PySCF. The Flatiron Institute is a division of the Simons Foundation. M.B. acknowledges support from the Departamento de Educación of the Basque Government through a Ph.D. grant as well as from Euskampus and the DIPC at the initial stages of his work. J.C. was supported by the Center for Molecular Magnetic Quantum Materials (M2QM), an Energy Frontier Research Center funded by the U.S. Department of Energy, Office of Science, Basic Energy Sciences under Award No. DE-SC0019330. J.J.E. acknowledges financial support from the Alexander von Humboldt Foundation and the Independent Research Fund Denmark. M.R.H. and H.Q.P. were partially supported by the U.S. Department of Energy, Office of Science, Basic Energy Sciences, Division of Chemical Sciences, Geosciences and Biosciences under Award No. DE-FG02-17ER16362 while working in the group of Laura Gagliardi at the University of Minnesota. P.K. acknowledges financial support from the Fellows Gipuzkoa program of the Gipuzkoako Foru Aldundia through the FEDER funding scheme of the European Union. S.L. was supported by the Academy of Finland (Suomen Akatemia) through Project No. 311149. A.P. thanks the Swiss NSF for the support provided through the Early Postdoc Mobility program

(Project No. P2ELP2_175281). H.F.S. acknowledges the financial support from the European Union via Marie Skłodowska-Curie Grant Agreement No. 754388 and LMUexcellent within the German Excellence Initiative (Grant No. ZUK22). S.B. and J.E.T.S. gratefully acknowledge support from a fellowship through The Molecular Sciences Software Institute under NSF Grant No. ACI-1547580. S.S. acknowledges support of the NSF (Grant No. CHE-1800584). S.U. acknowledges the support of the NSF (Grant No. CHE-1762337). J.M.Y. acknowledges support of the National Science Foundation Graduate Research Fellowship Program. N.S.B. acknowledges funding and support from St. John's College, Cambridge.

DATA AVAILABILITY

The data that support the findings of this study are available within the article and/or from the corresponding author upon reasonable request.

REFERENCES

- Q. Sun, T. C. Berkelbach, N. S. Blunt, G. H. Booth, S. Guo, Z. Li, J. Liu, J. D. McClain, E. R. Sayfutyarova, S. Sharma, S. Wouters, and G. K. L. Chan, *Wiley Interdiscip. Rev.: Comput. Mol. Sci.* **8**, e1340 (2018).
- Python-based Simulations of Chemistry Framework, 2020, <https://github.com/PySCF/PySCF>; accessed on 16 April 2020.
- PySCF: Python-based Simulations of Chemistry Framework, 2020, <https://pypi.org/project/PySCF/>; accessed on 16 April 2020.
- Python-based Simulations of Chemistry Framework, 2020, <https://anaconda.org/PySCF/PySCF>; accessed on 16 April 2020.
- G. Chen, P. Chen, C.-Y. Hsieh, C.-K. Lee, B. Liao, R. Liao, W. Liu, J. Qiu, Q. Sun, J. Tang, R. Zemel, and S. Zhang, *arXiv:1906.09427* [cs.LG] (2019).
- C. Lu, Q. Liu, Q. Sun, C.-Y. Hsieh, S. Zhang, L. Shi, and C.-K. Lee, *arXiv:1910.13551* [physics.chem-ph] (2019).
- S. Dick and M. Fernandez-Serra, *chemRxiv:9947312* (2019).
- H. Ji and Y. Jung, *J. Chem. Phys.* **148**, 241742 (2018).
- J. Hermann, Z. Schätzle, and F. Noé, *arXiv:1909.08423* [physics.comp-ph] (2019).
- J. Han, L. Zhang, and W. E, *J. Comput. Phys.* **399**, 108929 (2019).
- J. Cartus, <https://github.com/jcartus/SCFInitialGuess>; accessed on 21 February 2020.
- D. Pfau, J. S. Spencer, A. G. d. G. Matthews, and W. M. C. Foulkes, *arXiv:1909.02487* [physics.chem-ph] (2019).
- K. Choo, A. Mezzacapo, and G. Carleo, *arXiv:1909.12852* [physics.comp-ph] (2019).
- J. R. McClean, K. J. Sung, I. D. Kivlichan, Y. Cao, C. Dai, E. S. Fried, C. Gidney, B. Gimby, P. Gokhale, T. Häner, T. Hardikar, V. Havlíček, O. Higgott, C. Huang, J. Izaac, Z. Jiang, X. Liu, S. McArdle, M. Neeley, T. O'Brien, B. O'Gorman, I. Ozfidan, M. D. Radin, J. Romero, N. Rubin, N. P. D. Sawaya, K. Setia, S. Sim, D. S. Steiger, M. Steudtner, Q. Sun, W. Sun, D. Wang, F. Zhang, and R. Babush, "OpenFermion: the electronic structure package for quantum computers," *Quantum Sci. Technol.* **5**, 3 (2020).
- H. Abraham, I. Y. Akhalwaya, G. Aleksandrowicz, T. Alexander, G. Alexandrowics, E. Arbel, A. Asfaw, C. Azaustre, AzizNgoueya, P. Barkoutsos, G. Barron, L. Bello, Y. Ben-Haim, D. Bevenius, L. S. Bishop, S. Bosch, S. Bravyi, D. Bucher, F. Cabrera, P. Calpin, L. Capelluto, J. Carballo, G. Carrascal, A. Chen, C.-F. Chen, R. Chen, J. M. Chow, C. Claus, C. Claus, A. J. Cross, A. W. Cross, S. Cross, J. Cruz-Benito, C. Culver, A. D. Córcoles-Gonzales, S. Dague, T. E. Dandachi, M. Dartailh, DavideFr, A. R. Davila, D. Ding, J. Doi, E. Drechsler, E. Dumitrescu, K. Dumon, I. Duran, K. El-Safty, E. Eastman, P. Eendebak, D. Egger, M. Everitt, P. M. Fernández, A. H. Ferrera, A. Frisch, A. Fuhrer, M. George, J. Gacon, Gadi, B. G. Gago, J. M. Gambetta, A. Gammanpila, L. García, S. Garion, J. Gomez-Mosquera, S. de la Puente González, I. Gould, D. Greenberg, D. Grinko, W. Guan, J. A. Gunnels, I. Haide, I. Hamamura, V. Havlicek, J. Hellmers,

- L. Herok, S. Hillmich, H. Horii, C. Howington, S. Hu, W. Hu, H. Imai, T. Imamichi, K. Ishizaki, R. Iten, T. Itoko, A. Javadi-Abhari, K. Johns, T. Kachmann, N. Kanazawa, A. Karazeev, P. Kassebaum, S. King, Knabberjoe, A. Kovyshin, V. Krishnan, K. Krsulich, G. Kus, R. LaRose, R. Lambert, J. Latone, S. Lawrence, D. Liu, P. Liu, Y. Maeng, A. Malyshev, J. Marecek, M. Marques, D. Mathews, A. Matsuo, D. T. McClure, C. McGarry, D. McKay, S. Meesala, M. Mevissen, A. Mezzacapo, R. Midha, Z. Minev, N. Moll, M. D. Mooring, R. Morales, N. Moran, P. Murali, J. O. Sokolov, D. Steenken, M. Stypulkoski, H. Takahashi, I. Tavernelli, C. Taylor, P. Taylour, S. Thomas, M. Tillet, M. Tod, E. de la Torre, K. Trabling, M. Treinish, W. Turner, Y. Vaknin, C. R. Valcarce, F. Varchon, A. C. Vazquez, D. Vogt-Lee, C. Vuillot, J. Weaver, R. Wiczorek, J. A. Wildstrom, R. Wille, E. Winston, J. J. Woehr, S. Woerner, R. Woo, C. J. Wood, R. Wood, S. Wood, J. Wootton, D. Yeralin, R. Young, J. Yu, C. Zachow, L. Zdanski, and C. Zoufal (2019), "Qiskit: An open-source framework for quantum computing," zenodo, <https://doi.org/10.5281/zenodo.2562111>.
- ¹⁶T. Yamazaki, S. Matsuura, A. Narimani, A. Saidmuradov, and A. Zaribafiyani, [arXiv:1806.01305](https://arxiv.org/abs/1806.01305) [quant-ph] (2018).
- ¹⁷PySCF: The Python-based Simulations of Chemistry Framework, 2020, <http://PySCF.org>; accessed on 16 April 2020.
- ¹⁸M. Valiev, E. J. Bylaska, N. Govind, K. Kowalski, T. P. Straatsma, H. J. J. Van Dam, D. Wang, J. Nieplocha, E. Apra, T. L. Windus, and W. A. de Jong, *Comput. Phys. Commun.* **181**, 1477 (2010).
- ¹⁹F. Furche, R. Ahlrichs, C. Hättig, W. Klopper, M. Sierka, and F. Weigend, *Wiley Interdiscip. Rev.: Comput. Mol. Sci.* **4**, 91 (2014).
- ²⁰Y. Shao, Z. Gan, E. Epifanovsky, A. T. B. Gilbert, M. Wormit, J. Kussmann, A. A. Lange, A. Behn, J. Deng, X. Feng, D. Ghosh, M. Goldey, P. R. Horn, L. D. Jacobson, I. Kaliman, R. Z. Khaliullin, T. Kus, A. Landau, J. Liu, E. I. Proynov, Y. M. Rhee, R. M. Richard, M. A. Rohrdanz, R. P. Steele, E. J. Sundstrom, H. L. Woodcock, P. M. Zimmerman, D. Zuev, B. Albrecht, E. Alguire, B. Austin, G. J. O. Beran, Y. A. Bernard, E. Berquist, K. Brandhorst, K. B. Bravaya, S. T. Brown, D. Casanova, C.-M. Chang, Y. Chen, S. H. Chien, K. D. Closser, D. L. Crittenden, M. Diedenhofen, R. A. DiStasio, H. Do, A. D. Dutoi, R. G. Edgar, S. Fatehi, L. Fusti-Molnar, A. Ghysels, A. Golubeva-Zadorozhnyaya, J. Gomes, M. W. D. Hanson-Heine, P. H. P. Harbach, A. W. Hauser, E. G. Hohenstein, Z. C. Holden, T.-C. Jagau, H. Ji, B. Kaduk, K. Khistyayev, J. Kim, J. Kim, R. A. King, P. Klunzinger, D. Kosenkov, T. Kowalczyk, C. M. Krauter, K. U. Lao, A. D. Laurent, K. V. Lawler, S. V. Levchenko, C. Y. Lin, F. Liu, E. Livshits, R. C. Lochan, A. Luenser, P. Manohar, S. F. Manzer, S.-P. Mao, N. Mardirossian, A. V. Marenich, S. A. Maurer, N. J. Mayhall, E. Neuscamman, C. M. Oana, R. Olivares-Amaya, D. P. O'Neill, J. A. Parkhill, T. M. Perrine, R. Peverati, A. Prociuk, D. R. Rehn, E. Rosta, N. J. Russ, S. M. Sharada, S. Sharma, D. W. Small, A. Sodt, T. Stein, D. Stück, Y.-C. Su, A. J. W. Thom, T. Tsuchimochi, V. Vanovschi, L. Vogt, O. Vydrov, T. Wang, M. A. Watson, J. Wenzel, A. White, C. F. Williams, J. Yang, S. Yeganeh, S. R. Yost, Z.-Q. You, I. Y. Zhang, X. Zhang, Y. Zhao, B. R. Brooks, G. K. L. Chan, D. M. Chipman, C. J. Cramer, W. A. Goddard, M. S. Gordon, W. J. Hehre, A. Klamt, H. F. Schaefer, M. W. Schmidt, C. D. Sherrill, D. G. Truhlar, A. Warshel, X. Xu, A. Aspuru-Guzik, R. Baer, A. T. Bell, N. A. Besley, J.-D. Chai, A. Dreuw, B. D. Dunietz, T. R. Furlani, S. R. Gwaltney, C.-P. Hsu, Y. Jung, J. Kong, D. S. Lambrecht, W. Liang, C. Ochsenfeld, V. A. Rassolov, L. V. Slipchenko, J. E. Subotnik, T. Van Voorhis, J. M. Herbert, A. I. Krylov, P. M. W. Gill, and M. Head-Gordon, *Mol. Phys.* **113**, 184 (2015).
- ²¹R. M. Parrish, L. A. Burns, D. G. A. Smith, A. C. Simmonett, A. E. DePrince, E. G. Hohenstein, U. Bozkaya, A. Y. Sokolov, R. Di Remigio, R. M. Richard, J. F. Gonthier, A. M. James, H. R. McAlexander, A. Kumar, M. Saitow, X. Wang, B. P. Pritchard, P. Verma, H. F. Schaefer, K. Patkowski, R. A. King, E. F. Valeev, F. A. Evangelista, J. M. Turney, T. D. Crawford, and C. D. Sherrill, *J. Chem. Theory Comput.* **13**, 3185 (2017).
- ²²G. Kresse and J. Furthmüller, *Phys. Rev. B* **54**, 11169 (1996).
- ²³G. Kresse and D. Joubert, *Phys. Rev. B* **59**, 1758 (1999).
- ²⁴J. Enkovaara, C. Rostgaard, J. J. Mortensen, J. Chen, M. Dulak, L. Ferighi, J. Gavnholt, C. Glinsvad, V. Haikola, H. A. Hansen, H. H. Kristoffersen, M. Kuisma, A. H. Larsen, L. Lehtovaara, M. Ljungberg, O. Lopez-Acevedo, P. G. Moses, J. Ojanen, T. Olsen, V. Petzold, N. A. Romero, J. Stausholm-Møller, M. Strange, G. A. Tritsaridis, M. Vanin, M. Walter, B. Hammer, H. Häkkinen, G. K. H. Madsen, R. M. Nieminen, J. K. Nørskov, M. Puska, T. T. Rantala, J. Schiøtz, K. S. Thygesen, and K. W. Jacobsen, *J. Phys. Condens. Matter* **22**, 253202 (2010).
- ²⁵P. Giannozzi, O. Andreussi, T. Brumme, O. Bunau, M. Buongiorno Nardelli, M. Calandra, R. Car, C. Cavazzoni, D. Ceresoli, M. Cococcioni, N. Colonna, I. Carnimeo, A. Dal Corso, S. de Gironcoli, P. Delugas, R. A. DiStasio, A. Ferretti, A. Floris, G. Fratesi, G. Fugallo, R. Gebauer, U. Gerstmann, F. Giustino, T. Gorni, J. Jia, M. Kawamura, H.-Y. Ko, A. Kokalj, E. Küçükbenli, M. Lazzeri, M. Marsili, N. Marzari, F. Mauri, N. L. Nguyen, H.-V. Nguyen, A. Otero-de-la-Roza, L. Paulatto, S. Poncè, D. Rocca, R. Sabatini, B. Santra, M. Schlipf, A. P. Seitsonen, A. Smogunov, I. Timrov, T. Thonhauser, P. Umari, N. Vast, X. Wu, and S. Baroni, *J. Phys. Condens. Matter* **29**, 465901 (2017).
- ²⁶J. VandeVondele, M. Krack, F. Mohamed, M. Parrinello, T. Chassaing, and J. Hutter, *Comput. Phys. Commun.* **167**, 103 (2005).
- ²⁷J. McClain, Q. Sun, G. K.-L. Chan, and T. C. Berkelbach, *J. Chem. Theory Comput.* **13**, 1209 (2017).
- ²⁸Q. Sun, T. C. Berkelbach, J. D. McClain, and G. K.-L. Chan, *J. Chem. Phys.* **147**, 164119 (2017).
- ²⁹S. Sharma and G. K.-L. Chan, *J. Chem. Phys.* **136**, 124121 (2012).
- ³⁰S. Wouters, W. Poelmans, P. W. Ayers, and D. Van Neck, *Comput. Phys. Commun.* **185**, 1501 (2014).
- ³¹S. Sharma, A. A. Holmes, G. Jeanmairet, A. Alavi, and C. J. Umrigar, *J. Chem. Theory Comput.* **13**, 1595 (2017).
- ³²J. E. T. Smith, B. Mussard, A. A. Holmes, and S. Sharma, *J. Chem. Theory Comput.* **13**, 5468 (2017).
- ³³A. A. Holmes, N. M. Tubman, and C. J. Umrigar, *J. Chem. Theory Comput.* **12**, 3674 (2016).
- ³⁴J. Li, M. Otten, A. A. Holmes, S. Sharma, and C. J. Umrigar, *J. Chem. Phys.* **149**, 214110 (2018).
- ³⁵G. H. Booth, S. D. Smart, and A. Alavi, *Mol. Phys.* **112**, 1855 (2014).
- ³⁶U. Bozkaya, J. M. Turney, Y. Yamaguchi, H. F. Schaefer, and C. D. Sherrill, *J. Chem. Phys.* **135**, 104103 (2011).
- ³⁷R. Seeger and J. A. Pople, *J. Chem. Phys.* **66**, 3045 (1977).
- ³⁸C. Van Wüllen, *J. Comput. Chem.* **23**, 779 (2002).
- ³⁹S. Lehtola, C. Steigemann, M. J. T. Oliveira, and M. A. L. Marques, *SoftwareX* **7**, 1 (2018).
- ⁴⁰U. Ekström, L. Visscher, R. Bast, A. J. Thorvaldsen, and K. Ruud, *J. Chem. Theory Comput.* **6**, 1971 (2010).
- ⁴¹O. A. Vydrov and T. Van Voorhis, *J. Chem. Phys.* **133**, 244103 (2010).
- ⁴²Q. Sun, *J. Comput. Chem.* **36**, 1664 (2015).
- ⁴³R. A. Friesner, *Chem. Phys. Lett.* **116**, 39 (1985).
- ⁴⁴F. Neese, F. Wennmohs, A. Hansen, and U. Becker, *Chem. Phys.* **356**, 98 (2009).
- ⁴⁵R. Izsák and F. Neese, *J. Chem. Phys.* **135**, 144105 (2011).
- ⁴⁶S. Goedecker, M. Teter, and J. Hutter, *Phys. Rev. B* **54**, 1703 (1996).
- ⁴⁷J. Hutter, M. Iannuzzi, F. Schiffmann, and J. VandeVondele, *Wiley Interdiscip. Rev.: Comput. Mol. Sci.* **4**, 15 (2014).
- ⁴⁸C. Angeli, R. Cimrighia, S. Evangelisti, T. Leininger, and J.-P. Malrieu, *J. Chem. Phys.* **114**, 10252 (2001).
- ⁴⁹S. Guo, M. A. Watson, W. Hu, Q. Sun, and G. K.-L. Chan, *J. Chem. Theory Comput.* **12**, 1583 (2016).
- ⁵⁰S. R. Langhoff and E. R. Davidson, *Int. J. Quantum Chem.* **8**, 61 (1974).
- ⁵¹J. A. Pople, R. Seeger, and R. Krishnan, *Int. J. Quantum Chem.* **12**, 149 (1977).
- ⁵²P. J. Knowles and N. C. Handy, *Chem. Phys. Lett.* **111**, 315 (1984).
- ⁵³J. Olsen, P. Jørgensen, and J. Simons, *Chem. Phys. Lett.* **169**, 463 (1990).
- ⁵⁴H. Sekino and R. J. Bartlett, *Int. J. Quantum Chem.* **26**, 255 (1984).
- ⁵⁵A. C. Scheiner, G. E. Scuseria, J. E. Rice, T. J. Lee, and H. F. Schaefer, *J. Chem. Phys.* **87**, 5361 (1987).
- ⁵⁶G. E. Scuseria, C. L. Janssen, and H. F. Schaefer, *J. Chem. Phys.* **89**, 7382 (1988).

- ⁵⁷K. Raghavachari, G. W. Trucks, J. A. Pople, and M. Head-Gordon, *Chem. Phys. Lett.* **157**, 479 (1989).
- ⁵⁸E. A. Salter, G. W. Trucks, and R. J. Bartlett, *J. Chem. Phys.* **90**, 1752 (1989).
- ⁵⁹G. E. Scuseria, *J. Chem. Phys.* **94**, 442 (1991).
- ⁶⁰M. Nooijen and R. J. Bartlett, *J. Chem. Phys.* **102**, 3629 (1995).
- ⁶¹H. Koch, A. Sánchez de Merás, T. Helgaker, and O. Christiansen, *J. Chem. Phys.* **104**, 4157 (1996).
- ⁶²M. Musial, S. A. Kucharski, and R. J. Bartlett, *J. Chem. Phys.* **118**, 1128 (2003).
- ⁶³A. I. Krylov, *Acc. Chem. Res.* **39**, 83 (2006).
- ⁶⁴H. J. Werner and P. J. Knowles, *J. Chem. Phys.* **82**, 5053 (1985).
- ⁶⁵H. J. A. Jensen, P. Jorgensen, and H. Ågren, *J. Chem. Phys.* **87**, 451 (1987).
- ⁶⁶J. Schirmer, *Phys. Rev. A* **26**, 2395 (1982).
- ⁶⁷J. Schirmer, L. S. Cederbaum, and O. Walter, *Phys. Rev. A* **28**, 1237 (1983).
- ⁶⁸J. Schirmer and A. B. Trofimov, *J. Chem. Phys.* **120**, 11449 (2004).
- ⁶⁹A. Dreuw and M. Wormit, *Wiley Interdiscip. Rev.: Comput. Mol. Sci.* **5**, 82 (2015).
- ⁷⁰S. Banerjee and A. Y. Sokolov, *J. Chem. Phys.* **151**, 224112 (2019).
- ⁷¹L. Hedin, *Phys. Rev.* **139**, A796 (1965).
- ⁷²F. Aryasetiawan and O. Gunnarsson, *Rep. Prog. Phys.* **61**, 237 (1998); [arXiv:9712013](https://arxiv.org/abs/9712013) [cond-mat].
- ⁷³X. Ren, P. Rinke, V. Blum, J. Wierfink, A. Tkatchenko, A. Sanfilippo, K. Reuter, and M. Scheffler, *New J. Phys.* **14**, 053020 (2012); [arXiv:1201.0655](https://arxiv.org/abs/1201.0655).
- ⁷⁴J. Wilhelm, M. Del Ben, and J. Hutter, *J. Chem. Theory Comput.* **12**, 3623 (2016).
- ⁷⁵J. F. Stanton and J. Gauss, *Theor. Chem. Acc.* **93**, 303 (1996).
- ⁷⁶J. C. Saeh and J. F. Stanton, *J. Chem. Phys.* **111**, 8275 (1999).
- ⁷⁷M. Motta, D. M. Ceperley, G. K.-L. Chan, J. A. Gomez, E. Gull, S. Guo, C. A. Jiménez-Hoyos, T. N. Lan, J. Li, F. Ma, A. J. Millis, N. V. Prokofev, U. Ray, G. E. Scuseria, S. Sorella, E. M. Stoudenmire, Q. Sun, I. S. Tupitsyn, S. R. White, D. Zgid, and S. Zhang, *Phys. Rev. X* **7**, 031059 (2017).
- ⁷⁸Q. Sun, J. Yang, and G. K.-L. Chan, *Chem. Phys. Lett.* **683**, 291 (2017).
- ⁷⁹R. E. Thomas, Q. Sun, A. Alavi, and G. H. Booth, *J. Chem. Theory Comput.* **11**, 5316 (2015).
- ⁸⁰X. Wang and T. C. Berkelbach, [arXiv:2001.11050](https://arxiv.org/abs/2001.11050) [cond-mat.mtrl-sci] (2020).
- ⁸¹Y. Gao, Q. Sun, J. M. Yu, M. Motta, J. McClain, A. F. White, A. J. Minnich, and G. K.-L. Chan, [arXiv:1910.02191](https://arxiv.org/abs/1910.02191) [cond-mat.mtrl-sci] (2019).
- ⁸²S. H. Vosko, L. Wilk, and M. Nusair, *Can. J. Phys.* **58**, 1200 (1980).
- ⁸³J. P. Perdew, K. Burke, and M. Ernzerhof, *Phys. Rev. Lett.* **77**, 3865 (1996).
- ⁸⁴L. Visscher, T. Enevoldsen, T. Saue, H. J. A. Jensen, and J. Oddershede, *J. Comput. Chem.* **20**, 1262 (1999).
- ⁸⁵T. Helgaker, M. Jaszuński, and K. Ruud, *Chem. Rev.* **99**, 293 (1999).
- ⁸⁶T. Enevoldsen, L. Visscher, T. Saue, H. J. A. Jensen, and J. Oddershede, *J. Chem. Phys.* **112**, 3493 (2000).
- ⁸⁷V. Sychrovský, J. Gräfenstein, and D. Cremer, *J. Chem. Phys.* **113**, 3530 (2000).
- ⁸⁸T. Helgaker, M. Watson, and N. C. Handy, *J. Chem. Phys.* **113**, 9402 (2000).
- ⁸⁹L. Cheng, Y. Xiao, and W. Liu, *J. Chem. Phys.* **130**, 144102 (2009).
- ⁹⁰G. Schreckenbach and T. Ziegler, *J. Phys. Chem. A* **101**, 3388 (1997).
- ⁹¹F. Neese, *J. Chem. Phys.* **115**, 11080 (2001).
- ⁹²Z. Rinkevicius, L. Telyatnyk, P. Salek, O. Vahtras, and H. Ågren, *J. Chem. Phys.* **119**, 10489 (2003).
- ⁹³P. Hrobárik, M. Repiský, S. Komorovský, V. Hrobáriková, and M. Kaupp, *Theor. Chem. Acc.* **129**, 715 (2011).
- ⁹⁴S. P. A. Sauer, J. Oddershede, and J. Geertsen, *Mol. Phys.* **76**, 445 (1992).
- ⁹⁵J. Gauss, K. Ruud, and T. Helgaker, *J. Chem. Phys.* **105**, 2804 (1996).
- ⁹⁶F. Neese, *J. Chem. Phys.* **118**, 3939 (2003).
- ⁹⁷A. V. Arbuznikov, J. Vaara, and M. Kaupp, *J. Chem. Phys.* **120**, 2127 (2004).
- ⁹⁸R. F. Curl, *Mol. Phys.* **9**, 585 (1965).
- ⁹⁹G. Tarczay, P. G. Szalay, and J. Gauss, *J. Phys. Chem. A* **114**, 9246 (2010).
- ¹⁰⁰T. A. Keith, *Chem. Phys.* **213**, 123 (1996).
- ¹⁰¹R. Cammi, *J. Chem. Phys.* **109**, 3185 (1998).
- ¹⁰²M. R. Pederson and S. N. Khanna, *Phys. Rev. B* **60**, 9566 (1999).
- ¹⁰³F. Neese, *J. Chem. Phys.* **127**, 164112 (2007).
- ¹⁰⁴S. Schmitt, P. Jost, and C. van Wüllen, *J. Chem. Phys.* **134**, 194113 (2011).
- ¹⁰⁵F. London, *J. Phys. Radium* **8**, 397 (1937).
- ¹⁰⁶R. Ditchfield, *Mol. Phys.* **27**, 789 (1974).
- ¹⁰⁷H. M. Petrilli, P. E. Blöchl, P. Blaha, and K. Schwarz, *Phys. Rev. B* **57**, 14690 (1998).
- ¹⁰⁸S. Adiga, D. Aebi, and D. L. Bryce, *Can. J. Chem.* **85**, 496 (2007).
- ¹⁰⁹J. Autschbach, S. Zheng, and R. W. Schurko, *Concepts Magn. Reson., Part A* **36A**, 84 (2010).
- ¹¹⁰K. G. Dyall, *J. Chem. Phys.* **106**, 9618 (1997).
- ¹¹¹W. Kutzelnigg and W. Liu, *J. Chem. Phys.* **123**, 241102 (2005).
- ¹¹²W. Liu and D. Peng, *J. Chem. Phys.* **125**, 044102 (2006).
- ¹¹³M. Iliaš and T. Saue, *J. Chem. Phys.* **126**, 064102 (2007).
- ¹¹⁴L. Cheng and J. Gauss, *J. Chem. Phys.* **134**, 244112 (2011).
- ¹¹⁵G. Knizia, *J. Chem. Theory Comput.* **9**, 4834 (2013).
- ¹¹⁶A. E. Reed, R. B. Weinstock, and F. Weinhold, *J. Chem. Phys.* **83**, 735 (1985).
- ¹¹⁷Q. Sun and G. K.-L. Chan, *J. Chem. Theory Comput.* **10**, 3784 (2014).
- ¹¹⁸J. M. Foster and S. F. Boys, *Rev. Mod. Phys.* **32**, 300 (1960).
- ¹¹⁹C. Edmiston and K. Ruedenberg, *J. Chem. Phys.* **43**, S97 (1965).
- ¹²⁰J. Pipek and P. G. Mezey, *J. Chem. Phys.* **90**, 4916 (1989).
- ¹²¹W. D. Derricotte and F. A. Evangelista, *J. Chem. Theory Comput.* **13**, 5984 (2017).
- ¹²²Q. Sun, [arXiv:1610.08423](https://arxiv.org/abs/1610.08423) [physics.chem-ph] (2016).
- ¹²³G. Pizzi, V. Vitale, R. Arita, S. Blügel, F. Freimuth, G. Géranton, M. Gibertini, D. Gresch, C. Johnson, T. Koretsune, J. Ibañez-Azpiroz, H. Lee, J.-M. Lihm, D. Marchand, A. Marrazzo, Y. Mokrousov, J. I. Mustafa, Y. Nohara, Y. Nomura, L. Paulatto, S. Poncé, T. Ponweiser, J. Qiao, F. Thöle, S. S. Tsirkin, M. Wierzbowska, N. Marzari, D. Vanderbilt, I. Souza, A. A. Mostofi, and J. R. Yates, *J. Phys. Condens. Matter* **32**, 165902 (2020).
- ¹²⁴A. Damle, L. Lin, and L. Ying, *J. Chem. Theory Comput.* **11**, 1463 (2015).
- ¹²⁵A. Damle, L. Lin, and L. Ying, *J. Comput. Phys.* **334**, 1 (2017).
- ¹²⁶A. Klamt and G. Schüürmann, *J. Chem. Soc., Perkin Trans. 2* **1993**, 799.
- ¹²⁷E. Cancès, B. Mennucci, and J. Tomasi, *J. Chem. Phys.* **107**, 3032 (1997).
- ¹²⁸B. Mennucci, E. Cancès, and J. Tomasi, *J. Phys. Chem. B* **101**, 10506 (1997).
- ¹²⁹E. Cancès, Y. Maday, and B. Stamm, *J. Chem. Phys.* **139**, 054111 (2013).
- ¹³⁰F. Lipparini, B. Stamm, E. Cancès, Y. Maday, and B. Mennucci, *J. Chem. Theory Comput.* **9**, 3637 (2013).
- ¹³¹F. Lipparini, G. Scalmani, L. Lagardère, B. Stamm, E. Cancès, Y. Maday, J.-P. Piquemal, M. J. Frisch, and B. Mennucci, *J. Chem. Phys.* **141**, 184108 (2014).
- ¹³²B. Stamm, E. Cancès, F. Lipparini, and Y. Maday, *J. Chem. Phys.* **144**, 054101 (2016).
- ¹³³F. Lipparini and B. Mennucci, *J. Chem. Phys.* **144**, 160901 (2016).
- ¹³⁴Z. Li, S. Guo, Q. Sun, and G. K.-L. Chan, *Nat. Chem.* **11**, 1026 (2019).
- ¹³⁵M. Scheurer, P. Reinholdt, E. R. Kjellgren, J. M. Haugaard Olsen, A. Dreuw, and J. Kongsted, *J. Chem. Theory Comput.* **15**, 6154 (2019).
- ¹³⁶M. Scheurer (2019), “CPPE: C++ and python library for polarizable embedding,” zenodo. <https://doi.org/10.5281/zenodo.3345696>
- ¹³⁷W. Liu and D. Peng, *J. Chem. Phys.* **131**, 031104 (2009).
- ¹³⁸R. Flores-Moreno, R. J. Alvarez-Mendez, A. Vela, and A. M. Köster, *J. Comput. Chem.* **27**, 1009 (2006).
- ¹³⁹B. Mussard and S. Sharma, *J. Chem. Theory Comput.* **14**, 154 (2018).
- ¹⁴⁰L.-P. Wang and C. Song, *J. Chem. Phys.* **144**, 214108 (2016).
- ¹⁴¹J. Hermann (2020), “Pyberny,” zenodo. <https://doi.org/10.5281/zenodo.3695038>
- ¹⁴²S. Grimme, J. Antony, S. Ehrlich, and H. Krieg, *J. Chem. Phys.* **132**, 154104 (2010).
- ¹⁴³J. L. C. Sainz, A python wrapper for DFT-D3, <https://github.com/cuanto/libdftd3>; accessed on 21 February 2020.
- ¹⁴⁴D. A. Matthews, [arXiv:1607.00291](https://arxiv.org/abs/1607.00291) [cs.MS] (2016).
- ¹⁴⁵J. Huang, D. A. Matthews, and R. A. van de Geijn, [arXiv:1704.03092](https://arxiv.org/abs/1704.03092) [cs.MS] (2017).

- ¹⁴⁶D. Matthews, “TBLIS is a library and framework for performing tensor operations, especially tensor contraction, using efficient native algorithms,” <https://github.com/devinamathews/tblis>; accessed on 21 February 2020.
- ¹⁴⁷J. Kim, A. D. Baczewski, T. D. Beaudet, A. Benali, M. C. Bennett, M. A. Berrill, N. S. Blunt, E. J. L. Borda, M. Casula, D. M. Ceperley, S. Chiesa, B. K. Clark, R. C. Clay, K. T. Delaney, M. Dewing, K. P. Esler, H. Hao, O. Heinonen, P. R. C. Kent, J. T. Krogel, I. Kylänpää, Y. W. Li, M. G. Lopez, Y. Luo, F. D. Malone, R. M. Martin, A. Mathuriya, J. McMinis, C. A. Melton, L. Mitas, M. A. Morales, E. Neuscamman, W. D. Parker, S. D. Pineda Flores, N. A. Romero, B. M. Rubenstein, J. A. R. Shea, H. Shin, L. Shulenburger, A. F. Tillack, J. P. Townsend, N. M. Tubman, B. Van Der Goetz, J. E. Vincent, D. C. Yang, Y. Yang, S. Zhang, and L. Zhao, *J. Phys. Condens. Matter* **30**, 195901 (2018).
- ¹⁴⁸L. K. Wagner, K. Williams, S. Pathak, B. Busemeyer, J. N. B. Rodrigues, Y. Chang, A. Munoz, and C. Loring, Python library for real space quantum Monte Carlo, <https://github.com/WagnerGroup/pyqmc>; accessed on 21 February 2020.
- ¹⁴⁹L. K. Wagner, M. Bajdich, and L. Mitas, *J. Comput. Phys.* **228**, 3390 (2009).
- ¹⁵⁰J. S. Spencer, N. S. Blunt, S. Choi, J. Etrych, M.-A. Filip, W. M. C. Foulkes, R. S. T. Franklin, W. J. Handley, F. D. Malone, V. A. Neufeld, R. Di Remigio, T. W. Rogers, C. J. C. Scott, J. J. Shepherd, W. A. Vigor, J. Weston, R. Xu, and A. J. W. Thom, *J. Chem. Theory Comput.* **15**, 1728 (2019).
- ¹⁵¹D. V. Chulhai and J. D. Goodpaster, *J. Chem. Theory Comput.* **13**, 1503 (2017).
- ¹⁵²H. R. Petras, D. S. Graham, S. K. Ramadugu, J. D. Goodpaster, and J. J. Shepherd, *J. Chem. Theory Comput.* **15**, 5332 (2019).
- ¹⁵³J. D. Goodpaster, D. S. Graham, and D. V. Chulhai (2019), “Goodpaster/QSoME: Initial release,” zenodo. <https://doi.org/10.5281/zenodo.3356913>
- ¹⁵⁴M. R. Hermes and L. Gagliardi, *J. Chem. Theory Comput.* **15**, 972 (2019).
- ¹⁵⁵H. Q. Pham, M. R. Hermes, and L. Gagliardi, *J. Chem. Theory Comput.* **16**, 130 (2020).
- ¹⁵⁶X. Zhang and E. A. Carter, *J. Chem. Theory Comput.* **15**, 949 (2019).
- ¹⁵⁷Z.-H. Cui, T. Zhu, and G. K.-L. Chan, *J. Chem. Theory Comput.* **16**, 119 (2019).
- ¹⁵⁸T. Zhu, Z.-H. Cui, and G. K.-L. Chan, *J. Chem. Theory Comput.* **16**, 141 (2020).
- ¹⁵⁹L. Gagliardi, D. G. Truhlar, G. Li Manni, R. K. Carlson, C. E. Hoyer, and J. L. Bao, *Acc. Chem. Res.* **50**, 66 (2017).
- ¹⁶⁰J. J. Eriksen, PyMBE: A Many-Body Expanded Correlation Code by Janus Juul Eriksen, <https://gitlab.com/januseriksen/pymbe>; accessed on 21 February 2020.
- ¹⁶¹J. J. Eriksen, F. Lipparini, and J. Gauss, *J. Phys. Chem. Lett.* **8**, 4633 (2017); [arXiv:1708.02103](https://arxiv.org/abs/1708.02103).
- ¹⁶²J. J. Eriksen and J. Gauss, *J. Chem. Theory Comput.* **14**, 5180 (2018); [arXiv:1807.01328](https://arxiv.org/abs/1807.01328).
- ¹⁶³J. J. Eriksen and J. Gauss, *J. Chem. Theory Comput.* **15**, 4873 (2019); [arXiv:1905.02786](https://arxiv.org/abs/1905.02786).
- ¹⁶⁴J. J. Eriksen and J. Gauss, *J. Phys. Chem. Lett.* **10**, 7910 (2019); [arXiv:1910.03527](https://arxiv.org/abs/1910.03527).
- ¹⁶⁵S. Isakov, A. A. Rusakov, D. Zgid, and E. Gull, *Phys. Rev. B* **100**, 085112 (2019).
- ¹⁶⁶W. Li, C. Chen, D. Zhao, and S. Li, *Int. J. Quantum Chem.* **115**, 641 (2015).
- ¹⁶⁷W. Li, Z. Ni, and S. Li, *Mol. Phys.* **114**, 1447 (2016).
- ¹⁶⁸G. F. von Rudorff and O. A. von Lilienfeld, “Alchemical perturbation density functional theory,” *Phys. Rev. Research* **2**, 023220 (2020).
- ¹⁶⁹G. F. Von Rudorff and O. A. Von Lilienfeld, *J. Phys. Chem. B* **123**, 10073 (2019).
- ¹⁷⁰P. Koval, M. Barbry, and D. Sánchez-Portal, *Comput. Phys. Commun.* **236**, 188 (2019).
- ¹⁷¹S. Schwalbe, L. Fiedler, T. Hahn, K. Trepte, J. Kraus, and J. Kortus, [arXiv:1905.02631](https://arxiv.org/abs/1905.02631) [physics.comp-ph] (2019).
- ¹⁷²M. F. Herbst, A. Dreuw, and J. E. Avery, *J. Chem. Phys.* **149**, 084106 (2018).
- ¹⁷³Z. Rinkevicius, X. Li, O. Vahtras, K. Ahmadzadeh, M. Brand, M. Ringholm, N. H. List, M. Scheurer, M. Scott, A. Dreuw, and P. Norman, *Wiley Interdiscip. Rev.: Comput. Mol. Sci.* e1457 (2019).
- ¹⁷⁴M. F. Herbst, M. Scheurer, T. Fransson, D. R. Rehn, and A. Dreuw, *Wiley Interdiscip. Rev.: Comput. Mol. Sci.* e1462 (2020).